

STE Manual Solution

Practical No.1: Write problem statement to define the project title with bounded scope of the project.

Conclusion : Successfully implemented problem statement to define the project title with bounded scope of the project.

Practical related questions

1. Write problem statement to define online shopping with bounded scope of the project.

Answer :

Problem Statement: Online Shopping System (Bounded Scope)

The **Online Shopping System** is a web-based application designed to provide customers with a convenient platform to browse products, add items to a shopping cart, and place orders online. The system aims to simplify the shopping experience by allowing users to purchase products from anywhere without visiting a physical store.

Bounded Scope of the Project:

- User registration and secure login.
- Browse products by category and view product details.
- Search for products using keywords.
- Add, update, or remove products from the shopping cart.
- Place orders and view order history.
- Admin can manage products, categories, inventory, and customer orders.

Out of Scope:

- Integration with real payment gateways (only simulated payment, if included).
- Live shipment tracking and logistics management.
- Multi-vendor marketplace functionality.
- AI-based product recommendations.
- Customer support chat and return/refund processing.

This project focuses on providing the core functionalities of an online shopping platform while keeping the implementation simple, manageable, and suitable for academic purposes.

2. Write problem statement to define Online Library Management System with bounded scope of the project.

Answer :

Problem Statement: Online Library Management System (Bounded Scope)

The **Online Library Management System** is a web-based application designed to automate the management of library operations. It enables librarians to maintain book records efficiently and allows members to search, borrow, and return books through a centralized system. The project aims to reduce manual record-keeping, improve accuracy, and make library services more accessible.

Bounded Scope of the Project:

- User registration and secure login for librarians and members.
- Add, update, delete, and manage book records.
- Search books by title, author, category, or ISBN.
- Issue and return books while maintaining availability status.
- View borrowing history and due dates.
- Admin/Librarian can manage users, books, and issue/return records.

Out of Scope:

- Integration with external digital libraries or e-book platforms.
- Online payment for fines.
- Barcode/RFID-based book tracking.
- AI-based book recommendation system.
- Multi-library network management and inter-library book transfers.

This project focuses on the essential functions of a library management system, making it suitable for academic use while ensuring efficient book and user management.

3. What Problem Statement and Scope of Project?

Answer :

Problem Statement

A **Problem Statement** is a clear and concise description of the issue or challenge that a project aims to solve. It explains **what the problem is, why it exists, who is affected, and why a solution is needed**. It serves as the foundation for defining the project's objectives and requirements.

Example:

Many libraries still maintain book records manually, which leads to errors, time-consuming searches, and inefficient management. An Online Library Management System is needed to automate these operations and improve efficiency.

Scope of Project

The **Scope of a Project** defines the **boundaries of the project**. It specifies **what features and functionalities are included (in scope) and what is excluded (out of scope)**. A well-defined scope helps prevent unnecessary work and ensures the project is completed within the planned time and resources.

Example (Online Library Management System):

In Scope:

- User registration and login
- Book management (add, update, delete)
- Search books
- Issue and return books
- View borrowing history

Out of Scope:

- Online fine payment
 - AI-based book recommendations
 - RFID/barcode integration
 - Multi-library network management
-

Practical No.2: Select relevant process model to define activities and related tasks set.

Conclusion : Successfully selected relevant process model to define activities and related tasks set.

Practical related questions

1. Which process model recommend for the Online Library Management System.

Answer :

The **Waterfall Process Model** is the most suitable process model for an **Online Library Management System**, especially for an academic project.

- The project requirements are **well-defined and stable**.
- Development follows a **step-by-step sequential approach**.
- Each phase is completed before moving to the next.
- It is **simple to understand, implement, and manage**.
- It is ideal for **small to medium-sized projects** with fixed requirements.

2. Write down the waterfall model activities and related tasks set for Hospital Patient Record Management System.

Answer :

1. Requirement Analysis : Gather requirements from hospital staff, doctors, and administrators; identify functional and non-functional requirements; prepare the Software Requirement Specification (SRS).

2. System Design : Design the system architecture, database schema, user interfaces, data flow diagrams (DFDs), and module structure for patient records, appointments, billing, and reports.

3. Implementation (Coding) : Develop modules such as patient registration, doctor management, appointment scheduling, patient records, billing, and report generation; integrate all modules.

4. Testing : Perform unit testing, integration testing, system testing, and user acceptance testing (UAT); identify and fix bugs; verify that all requirements are met.

5. Deployment : Install the software in the hospital environment, configure the database, migrate existing patient records (if required), and train hospital staff.

6. Maintenance : Correct defects, improve system performance, update security features, add new functionalities, and provide technical support after deployment.

Practical No.3: Gather application specific requirements for assimilate into RE (Requirement's engineering) Model.

Conclusion : Successfully gathered application specific requirements for assimilate into RE (Requirement's engineering) Model.

Practical related questions

1. Write the application-specific requirements for an Online Shopping system and organize them into a Requirements Engineering (RE) model.

Answer :

1. Functional Requirements

- Users shall be able to register and log in securely.
- Users shall be able to browse products by category.
- Users shall be able to search products by name or keyword.
- Users shall be able to view product details.
- Users shall be able to add, update, or remove items from the shopping cart.
- Users shall be able to place orders and view order history.
- Admin shall be able to add, edit, and delete products.
- Admin shall be able to manage categories, inventory, customers, and orders.

2. Non-Functional Requirements

- The system should provide a user-friendly interface.
- The system should respond quickly to user requests.
- User data should be secure through authentication and authorization.
- The system should be reliable and available whenever required.
- The application should be scalable to support an increasing number of users and products.
- The system should be compatible with major web browsers.

3. Requirements Engineering (RE) Model**a) Requirements Elicitation**

- Gather requirements from customers, administrators, and stakeholders through interviews, questionnaires, and observations.

b) Requirements Analysis

- Analyze and prioritize the collected requirements.
- Identify functional and non-functional requirements.
- Resolve any conflicts or ambiguities in the requirements.

c) Requirements Specification

- Document all requirements in a Software Requirements Specification (SRS) document.
- Clearly define system features, constraints, and user interactions.

d) Requirements Validation

- Review the requirements with stakeholders.
- Verify that the documented requirements are complete, consistent, correct, and feasible.

e) Requirements Management

- Track requirement changes throughout the project.
- Maintain version control and update documentation whenever requirements change.

2. Identify and document the requirements for an Online Library Management System and represent them using RE techniques.

Answer :

1. Functional Requirements

- Users shall be able to register and log in to the system.
- Members shall be able to search books by title, author, category, or ISBN.
- Librarians shall be able to add, update, and delete book records.
- Members shall be able to borrow and return books.
- The system shall maintain book availability status.
- Users shall be able to view their borrowing history and due dates.
- The system shall generate reports for issued, returned, and overdue books.

2. Non-Functional Requirements

- The system should provide a simple and user-friendly interface.
- The system should ensure data security through authentication and authorization.
- The system should provide fast response time for searches and transactions.
- The system should be reliable and available with minimal downtime.
- The system should support multiple users simultaneously.
- The application should be compatible with major web browsers.

Representation Using Requirements Engineering (RE) Techniques

a) Requirements Elicitation

- Collect requirements from librarians, students, faculty members, and administrators through interviews, questionnaires, and observation.

b) Requirements Analysis

- Analyze the collected requirements.
- Classify them into functional and non-functional requirements.
- Identify dependencies, priorities, and resolve conflicts.

c) Requirements Specification

- Prepare a Software Requirements Specification (SRS) document containing system features, constraints, user roles, and interface requirements.

d) Requirements Validation

- Review the requirements with stakeholders.
- Ensure the requirements are complete, correct, consistent, feasible, and testable.

e) Requirements Management

- Maintain and update requirements throughout the software development life cycle.
- Track requirement changes using version control and change management practices.

3. What is the importance of gathering application-specific requirements and how are they integrated into an RE model?

Answer :

Importance of Gathering Application-Specific Requirements and Their Integration into a Requirements Engineering (RE) Model

Application-specific requirements define the unique features, functions, and constraints of a particular software application. Gathering these requirements is essential because they ensure that the system meets the actual needs of users and stakeholders. Well-defined requirements reduce misunderstandings, minimize development errors, improve software quality, and help deliver a system that satisfies customer expectations. They also provide a clear foundation for system design, implementation, testing, and maintenance.

Integration into the Requirements Engineering (RE) Model

1. **Requirements Elicitation:** Collect application-specific requirements from users, customers, and stakeholders through interviews, questionnaires, observations, and discussions.
 2. **Requirements Analysis:** Analyze the gathered requirements, remove ambiguities, resolve conflicts, and classify them into functional and non-functional requirements.
 3. **Requirements Specification:** Document the validated requirements in a Software Requirements Specification (SRS) document, clearly defining the system's features, constraints, and expected behaviour.
 4. **Requirements Validation:** Review the documented requirements with stakeholders to ensure they are complete, accurate, consistent, feasible, and testable.
 5. **Requirements Management:** Monitor and manage requirement changes throughout the Software Development Life Cycle (SDLC), ensuring that updates are documented and communicated effectively.
-

Practical No.4: Prepare broad SRS (software requirement Specification) for the project.

Conclusion : Successfully prepared broad SRS (software requirement Specification) for the project.

Practical related questions

1. Prepare a broad Software Requirements Specification (SRS) for an Online Shopping system covering functional and non-functional requirements.

Answer :

Software Requirements Specification (SRS) for an Online Shopping System

1. Introduction

The **Online Shopping System** is a web-based application that enables customers to browse products, add items to a shopping cart, and place orders online. The system also provides an administration panel to manage products, customers, inventory, and orders.

2. Functional Requirements

User Module

- User registration and secure login.
- User profile management.
- Password reset functionality.

Product Module

- Browse products by category.
- Search products by name or keyword.
- View product details, price, and availability.

Shopping Cart Module

- Add products to the cart.
- Update product quantity.
- Remove products from the cart.
- View total order amount.

Order Management

- Place orders.
- View order history.
- Cancel orders before processing.
- Receive order confirmation.

Admin Module

- Add, edit, and delete products.
- Manage product categories.
- Update inventory.
- Manage customer accounts.
- View and process customer orders.

3. Non-Functional Requirements

- **Performance:** The system should respond to user requests within a few seconds.
- **Security:** User authentication, authorization, and password encryption should be implemented.
- **Reliability:** The system should provide accurate and consistent data with minimal downtime.
- **Usability:** The interface should be simple, intuitive, and user-friendly.
- **Scalability:** The system should support an increasing number of users and products.
- **Availability:** The application should be available 24×7 with regular backups.
- **Compatibility:** The system should work on major web browsers and mobile devices.
- **Maintainability:** The application should be easy to update and maintain.

4. System Constraints

- Requires an internet connection.
- Users must register before placing an order.
- Only administrators can manage products and inventory.

5. Assumptions

- Users have basic knowledge of using a web browser.
- The server and database are always available.
- Product information is maintained by the administrator.

6. Conclusion

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the Online Shopping System. It serves as a guideline for developers, testers, and stakeholders to ensure the system is developed according to user needs and project objectives.

2. Develop a structured SRS document for an Online Library Management System following IEEE 830 or similar standards.

Answer :

Software Requirements Specification (SRS) for an Online Library Management System (IEEE 830 Format)

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of the **Online Library Management System**. It serves as a reference for developers, testers, and stakeholders during the software development process.

1.2 Scope

The Online Library Management System is a web-based application that automates library operations such as book management, member registration, book issue/return, and report generation. It improves efficiency, reduces manual work, and provides quick access to library resources.

1.3 Definitions

- **Admin/Librarian:** User who manages books, members, and library operations.
- **Member:** Registered user who can search, borrow, and return books.
- **SRS:** Software Requirements Specification.

2. Overall Description

2.1 Product Perspective

The system is a standalone web application connected to a centralized database for storing books, member details, and transaction records.

2.2 Product Functions

- User registration and login.
- Book search and viewing.
- Book issue and return.
- Book availability management.
- Member management.
- Report generation.
- Admin management of books and users.

2.3 User Classes

- **Administrator**
- **Librarian**
- **Library Member**

2.4 Operating Environment

- Windows/Linux operating system.
- Web browser (Chrome, Edge, Firefox).
- MySQL or similar database.
- Internet/Intranet connection.

2.5 Constraints

- Internet connection is required.
- Only authorized users can access the system.
- Database backup should be maintained regularly.

2.6 Assumptions

- Users possess basic computer knowledge.
- The database server remains available during system operation.

3. Specific Requirements

3.1 Functional Requirements

- Users shall register and log in securely.
- Members shall search books by title, author, category, or ISBN.
- Librarians shall add, update, and delete book records.
- Members shall borrow and return books.
- The system shall maintain book availability status.
- Users shall view borrowing history and due dates.
- The system shall generate reports for issued, returned, and overdue books.

3.2 Non-Functional Requirements

- **Performance:** Fast response for searches and transactions.
- **Security:** Authentication, authorization, and encrypted passwords.
- **Reliability:** Accurate data with minimum downtime.
- **Usability:** Simple and user-friendly interface.
- **Scalability:** Support multiple users and increasing book records.
- **Availability:** System available during library working hours or 24×7 if hosted online.
- **Maintainability:** Easy to maintain and update.

3.3 External Interface Requirements

- **User Interface:** Responsive web pages for users and administrators.
- **Hardware Interface:** Standard desktop/laptop computers with internet access.
- **Software Interface:** Web server, database server, and web browser.
- **Communication Interface:** HTTP/HTTPS protocol.

4. Appendices

- Software Requirement Specification (SRS) document.
- Database schema.
- Use Case Diagram.
- Entity Relationship (ER) Diagram.
- Data Flow Diagram (DFD).

5. Conclusion

This SRS document follows the IEEE 830 structure and provides a comprehensive specification for the Online Library Management System. It defines the system's objectives, functional and non-functional requirements, constraints, interfaces, and assumptions, serving as a foundation for design, development, testing, and maintenance.

3. Explain the significance of a Software Requirements Specification (SRS) and how it supports the software development lifecycle.

Answer :

Significance of a Software Requirements Specification (SRS) and Its Role in the Software Development Life Cycle (SDLC)

A **Software Requirements Specification (SRS)** is a formal document that describes the complete requirements of a software system. It defines the system's functional and non-functional requirements, constraints, interfaces, and expected behaviour. The SRS acts as a contract between stakeholders, customers, developers, and testers, ensuring that everyone has a common understanding of the project requirements.

Significance of an SRS

1. **Provides Clear Requirements:** Clearly defines what the software should do and what is expected from the system.
2. **Improves Communication:** Serves as a common reference for customers, developers, testers, and project managers.
3. **Reduces Errors:** Helps identify ambiguities and inconsistencies before development begins, reducing costly mistakes.
4. **Supports Planning:** Assists in project estimation, scheduling, budgeting, and resource allocation.
5. **Acts as a Reference Document:** Guides developers during implementation and testers during software verification and validation.
6. **Facilitates Maintenance:** Makes future enhancements and modifications easier by providing complete system documentation.
7. **Ensures Quality:** Helps ensure that the final software meets user needs and project objectives.

How SRS Supports the Software Development Life Cycle (SDLC)

1. Requirement Analysis

The SRS captures and documents all user and system requirements, providing the foundation for the project.

2. System Design

Designers use the SRS to create the system architecture, database design, user interface, and module structure.

3. Implementation (Coding)

Developers refer to the SRS to implement all specified functionalities accurately.

4. Testing

Test engineers prepare test cases based on the requirements defined in the SRS to verify that the software behaves as expected.

5. Deployment

The SRS ensures that the deployed system satisfies all documented requirements and stakeholder expectations.

6. Maintenance

During maintenance, the SRS serves as a reference for fixing defects, updating features, and implementing future enhancements.

Conclusion

The **Software Requirements Specification (SRS)** is one of the most important documents in software engineering. It provides a clear description of system requirements and supports every phase of the Software Development Life Cycle (SDLC), resulting in better communication, improved software quality, reduced development risks, and successful project completion.

Practical No.5: Write use-cases and draw use-case diagram.

Conclusion : Successfully wrote use-cases and drew use-case diagram.

Practical related questions

1. Identify and document use-cases for an Online Shopping system, and create the corresponding use-case diagram.

Answer :

1. Actors

- **Customer** – Browses products, manages cart, and places orders.
- **Admin** – Manages products, categories, inventory, customers, and orders.

2. Use Cases

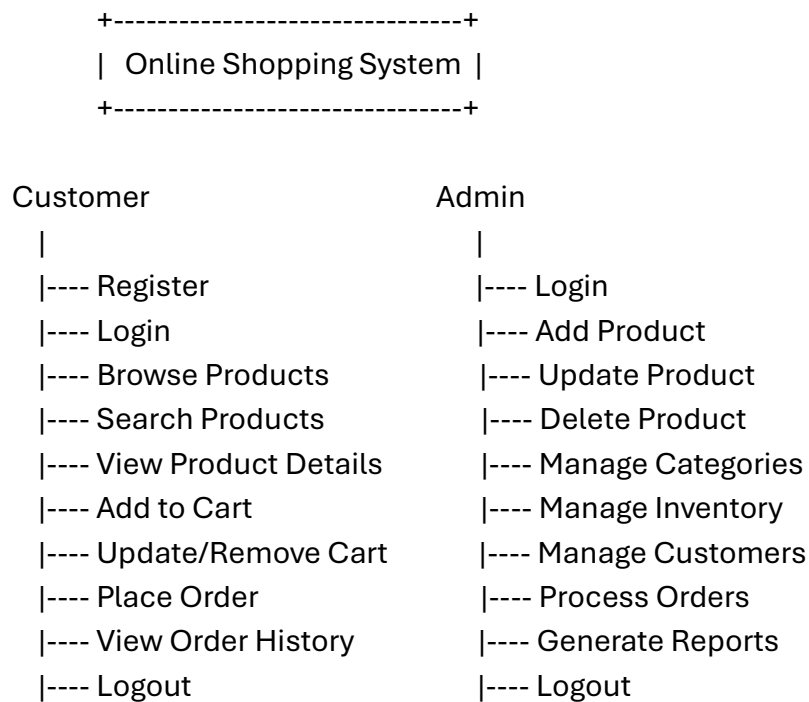
Customer Use Cases

- Register
- Login
- Browse Products
- Search Products
- View Product Details
- Add Product to Cart
- Update/Remove Cart Items
- Place Order
- View Order History
- Logout

Admin Use Cases

- Login
- Add Product
- Update Product
- Delete Product
- Manage Categories
- Manage Inventory
- Manage Customer Accounts
- Process Orders
- Generate Reports
- Logout

3. Use-Case Diagram (Text Representation)



4. Description

The **Customer** interacts with the system to register, log in, browse products, search for items, view product details, manage the shopping cart, place orders, and view order history.

The **Admin** logs into the system to manage products, categories, inventory, customer accounts, process orders, generate reports, and maintain the overall functionality of the Online Shopping System.

2. Develop use-case descriptions and draw a use-case diagram for an Online Library Management System.

Answer :

System

1. Actors

- **Member** – Searches, borrows, and returns books.
- **Librarian/Admin** – Manages books, members, and library transactions.

2. Use-Case Descriptions

Use Case 1: Register

- **Actor:** Member
- **Description:** A new user registers to access library services.
- **Precondition:** User is not already registered.
- **Postcondition:** A new member account is created.

Use Case 2: Login

- **Actor:** Member, Librarian
- **Description:** Users log in using valid credentials.
- **Precondition:** User account exists.
- **Postcondition:** User gains access to the system.

Use Case 3: Search Book

- **Actor:** Member
- **Description:** Member searches books by title, author, category, or ISBN.
- **Precondition:** User is logged in.
- **Postcondition:** Matching books are displayed.

Use Case 4: Borrow Book

- **Actor:** Member
- **Description:** Member borrows an available book.
- **Precondition:** Book is available.
- **Postcondition:** Book is issued and availability is updated.

Use Case 5: Return Book

- **Actor:** Member
- **Description:** Member returns a borrowed book.
- **Precondition:** Book has been issued to the member.
- **Postcondition:** Book status is updated as available.

Use Case 6: Manage Books

- **Actor:** Librarian/Admin
- **Description:** Add, update, or delete book records.
- **Precondition:** Librarian is logged in.
- **Postcondition:** Library database is updated.

Use Case 7: Manage Members

- **Actor:** Librarian/Admin
- **Description:** Add, update, or remove member records.
- **Precondition:** Librarian is logged in.
- **Postcondition:** Member information is updated.

Use Case 8: Generate Reports

- **Actor:** Librarian/Admin
- **Description:** Generate reports of issued, returned, and overdue books.
- **Precondition:** Library data is available.
- **Postcondition:** Reports are generated successfully.

3. Use-Case Diagram (Text Representation)

```
+-----+
| Online Library Management System |
+-----+
```

Member	Librarian/Admin
---- Register	---- Login
---- Login	---- Manage Books
---- Search Book	---- Manage Members
---- Borrow Book	---- Issue/Return Books
---- Return Book	---- Generate Reports
---- View Borrowing History	---- Logout
---- Logout	

4. Description

The **Member** can register, log in, search for books, borrow and return books, view borrowing history, and log out.

The **Librarian/Admin** can log in, manage books and member records, issue and receive returned books, generate library reports, and maintain the overall library system.

3. Explain the importance of use-cases and use-case diagrams in understanding system requirements and facilitating communication among stakeholders.

Answer :

Use cases and **use-case diagrams** are important tools in Requirements Engineering (RE) because they describe how users (actors) interact with a software system to achieve specific goals. They help developers, designers, testers, and stakeholders understand the system's functional requirements in a simple and visual manner.

Importance of Use Cases

1. Clearly describe the functional requirements of the system.
2. Identify the interactions between users and the system.
3. Help developers understand what features need to be implemented.
4. Assist testers in creating test cases based on user scenarios.
5. Reduce misunderstandings by providing detailed descriptions of system behaviour.

Importance of Use-Case Diagrams

1. Provide a visual representation of the system and its actors.
2. Show the relationships between users and system functionalities.
3. Help stakeholders easily understand the scope of the system.
4. Simplify complex requirements through graphical representation.
5. Improve communication among customers, developers, analysts, and project managers.

Role in Understanding System Requirements

- Identify all users (actors) interacting with the system.
- Define the services or functions provided by the system.
- Clarify the system boundaries and project scope.
- Ensure that all functional requirements are captured before development begins.

Role in Facilitating Communication Among Stakeholders

- Provide a common language that both technical and non-technical stakeholders can understand.
- Help gather feedback from customers and end users during the requirement analysis phase.
- Reduce ambiguity and improve collaboration among developers, testers, analysts, and clients.
- Ensure everyone has a shared understanding of the system's functionality and project objectives.

Conclusion

Use cases and use-case diagrams play a vital role in software development by clearly defining system functionality, improving requirement analysis, and enhancing communication among stakeholders. They provide a solid foundation for system design, implementation, testing, and maintenance, leading to the successful development of high-quality software systems.

Practical No.6: Draw the activity diagram to represent flow from one activity to another for software development.

Conclusion : Successfully drew the activity diagram to represent flow from one activity to another for software development.

Practical related questions

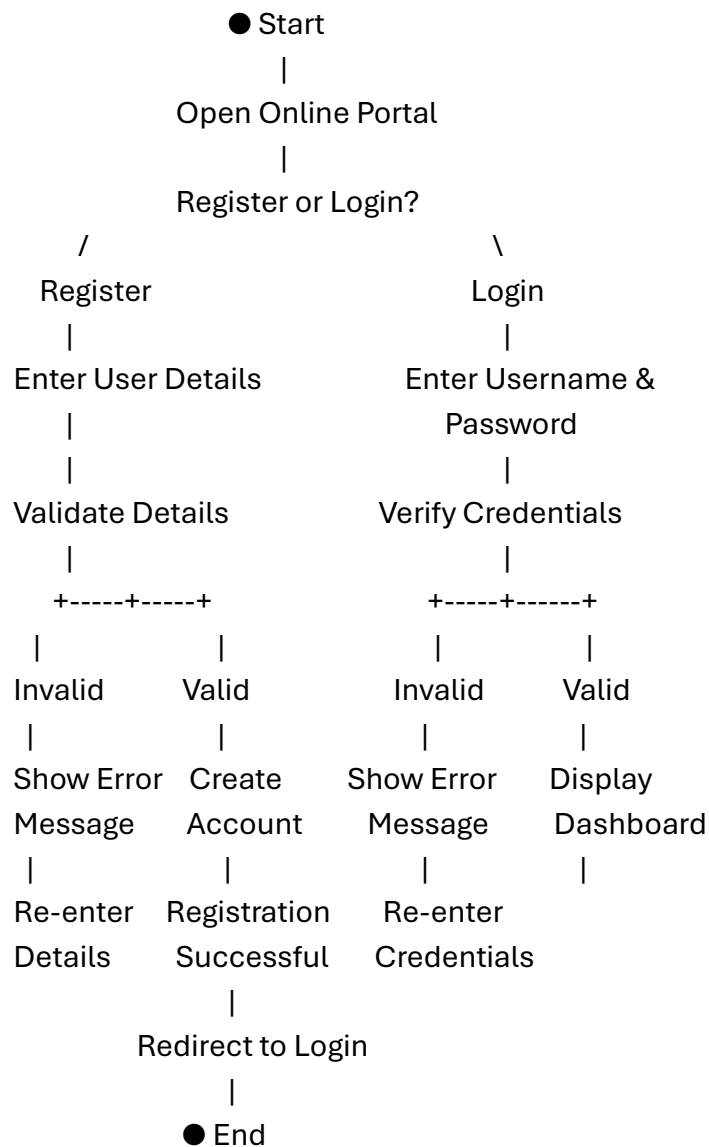
1. Develop an activity diagram illustrating the steps involved in user registration and login for an online portal.

Answer :

Activity Steps

1. Start
2. User opens the online portal.
3. User selects **Register** or **Login**.
4. **If Register is selected:**
 - Enter personal details (Name, Email, Password, etc.).
 - System validates the information.
 - If the details are valid, create a new user account.
 - Display "**Registration Successful**".
 - Redirect the user to the Login page.
5. **If Login is selected:**
 - Enter Username/Email and Password.
 - System verifies the credentials.
 - If credentials are valid, grant access and display the user dashboard.
 - If credentials are invalid, display an error message and ask the user to try again.
6. End.

Activity Diagram



Description

The activity diagram illustrates the workflow of user registration and login in an online portal. It begins when the user accesses the portal and chooses either to register or log in. During registration, the system validates the entered information before creating an account. During login, the system verifies the user's credentials and either grants access to the dashboard or displays an error message if the credentials are invalid. The process ends after successful registration or login.

2. What is an activity diagram used for in software engineering?

Answer :

An **Activity Diagram** is a UML (Unified Modeling Language) behavioral diagram used to represent the **workflow of activities** in a software system. It illustrates the sequence of actions, decisions, parallel processes, and the flow of control from the beginning to the end of a process.

Activity diagrams are widely used in software engineering to model workflows, business processes, and system behaviour. They improve communication among stakeholders, assist in system design and testing, and provide a clear understanding of how different activities are executed within a software system.

3. Name the key components of an activity diagram.

Answer :

The key components of an activity diagram are **Initial Node, Activity, Control Flow, Decision Node, Merge Node, Fork Node, Join Node, Final Node, and Swimlanes**.

Together, these elements provide a clear graphical representation of the workflow, making it easier to understand, design, and communicate software processes.

4. How does an activity diagram differ from a flowchart?

Answer :

An **Activity Diagram** is a UML-based behavioral diagram used to model workflows and interactions within a software system, while a **Flowchart** is a general diagram used to illustrate the sequence of steps in a process or algorithm. Activity diagrams are more suitable for software engineering because they support decision points, parallel execution, and responsibility modeling through swimlanes.

Practical No.7: Create DFDs (data flow diagram), Decision tables and E-R (entity-relationship) diagram.

Conclusion : Successfully created DFDs (data flow diagram), Decision tables and E-R (entity-relationship) diagram.

Practical related questions

1. What is a Data Flow Diagram (DFD)?

Answer :

A **Data Flow Diagram (DFD)** is a graphical representation of how data moves through a system. It illustrates the flow of data between **external entities, processes, data stores, and data flows**. A DFD helps in understanding how information is input, processed, stored, and output within a software system.

A **Data Flow Diagram (DFD)** is an important system analysis tool used to model the flow of data within a software system. It helps developers and stakeholders understand how data is processed, stored, and transferred, making it easier to design and develop efficient software systems.

2. Describe the main components of a Data Flow Diagram with examples.

Answer :

1. External Entity : Source or destination of data outside the system

Example : Customer, Admin

2. Process : Converts input data into output

Example : Process Order, Login Validation

3. Data Flow : Movement of data between components

Example : Order Details, Payment Information

4. Data Store : Stores system data

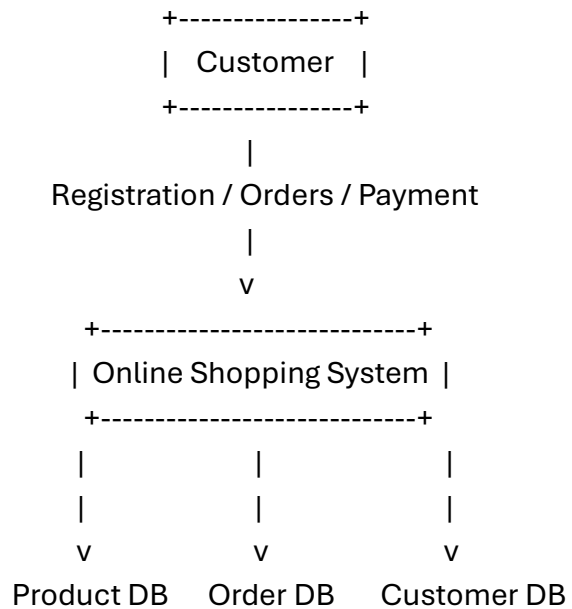
Example : Customer Database, Order Database

3. Create detailed DFDs, decision tables, and E-R diagrams for an Online Shopping System to show data flow, decision points, and entity relationships.

Answer :

1. Data Flow Diagram (DFD)

Level 0 (Context Diagram)



Level 1 DFD

Customer

|

| Registration/Login

v

+-----+

| 1. User Module |

+-----+

|

v

Customer Database

Customer

|

| Search Product

v

+-----+

| 2. Product Module |

+-----+

|

v

Product Database

Customer

|

| Add to Cart / Place Order

v

+-----+

| 3. Order Module |

+-----+

|

v

|

v

Order DB Payment Details

|

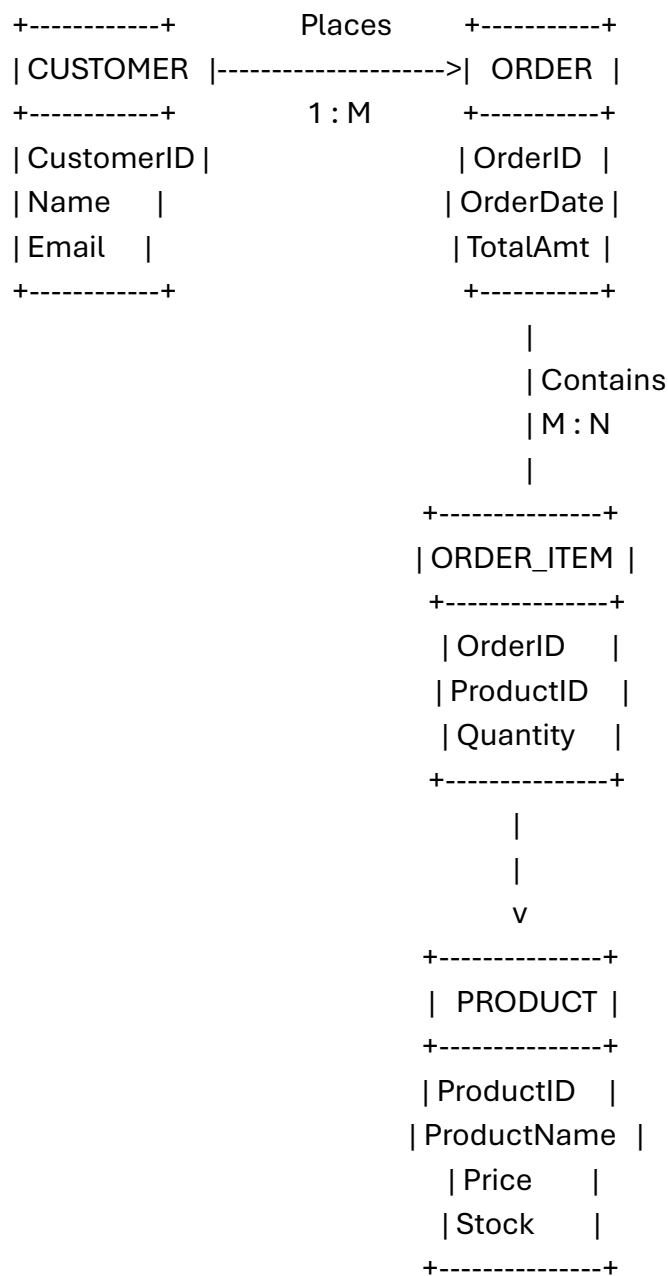
v

Order Confirmation

2. Decision Table

Condition	Rule 1	Rule 2	Rule 3	Rule 4
User Logged In?	Yes	Yes	No	No
Product Available?	Yes	No	Yes	No
Payment Successful?	Yes	No	–	–
Action				
Place Order	✓	X	X	X
Display Out of Stock	X	✓	X	X
Ask User to Login	X	X	✓	✓
Cancel Transaction	X	✓	✓	✓

3. Entity-Relationship (E-R) Diagram



Relationship Summary

- One **Customer** can place many **Orders** (1:M).
- One **Order** can contain many **Products**, and one **Product** can appear in many **Orders** (M:N), resolved using the **Order_Item** entity.
- Each **Product** contains information such as Product ID, Name, Price, and Stock Quantity.

Conclusion

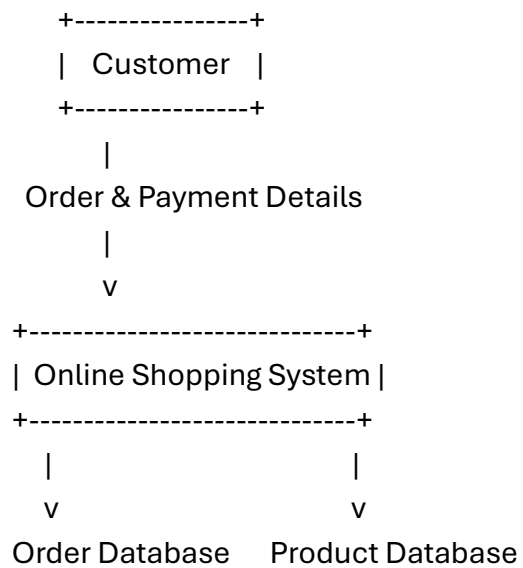
The **DFD** illustrates how data moves through the Online Shopping System, the **Decision Table** represents the business rules for order processing, and the **E-R Diagram** models the relationships among customers, orders, order items, and products. Together, these models provide a complete understanding of the system's data flow, decision logic, and database structure.

4. Develop DFDs and decision tables illustrating the order processing and payment approval steps, and draw an E-R diagram for customers, products, and orders.

Answer :

1. Data Flow Diagram (DFD)

Level 0 (Context Diagram)



Level 1 DFD

Customer

|

| Order Details

v

+-----+

| 1. Process Order |

+-----+

|

| Verify Product

v

Product Database

|

v

+-----+

| 2. Payment Approval |

+-----+

|

| Payment Status

v

+-----+

| 3. Generate Order |

| Confirmation |

+-----+

|

v

Order Database

|

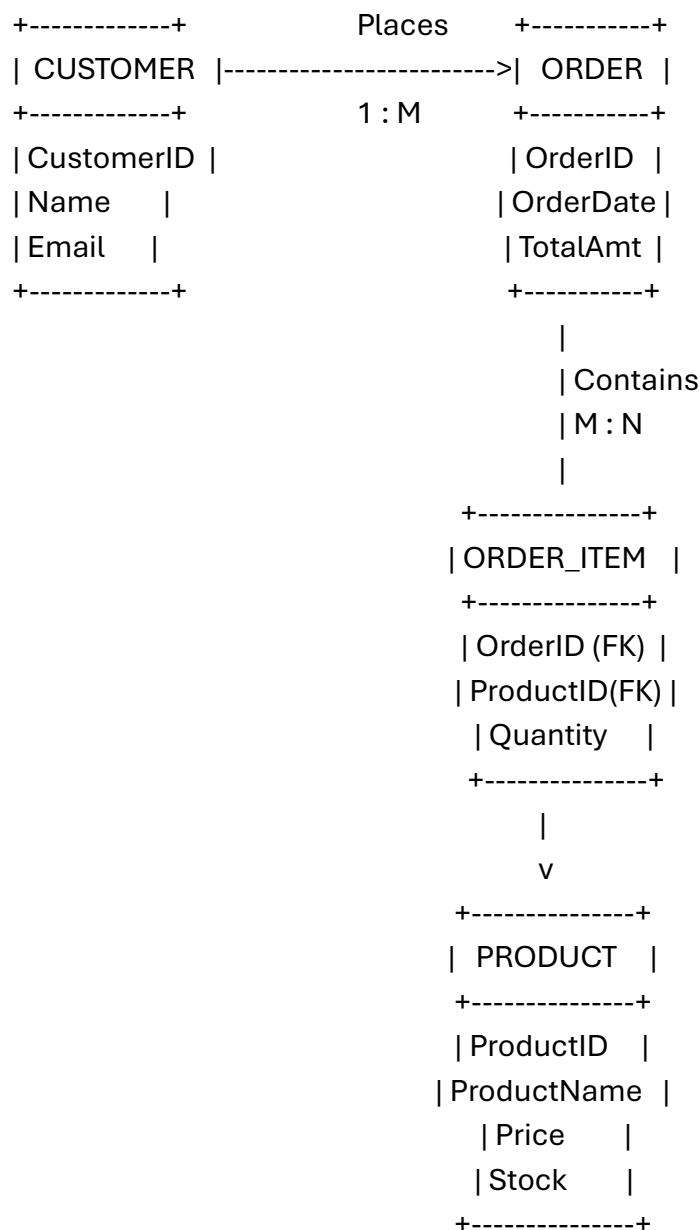
v

Customer

2. Decision Table (Order Processing & Payment Approval)

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Product Available?	Yes	Yes	No	No
Payment Successful?	Yes	No	Yes	No
Actions				
Approve Order	✓	✗	✗	✗
Update Order Database	✓	✗	✗	✗
Generate Invoice	✓	✗	✗	✗
Display Payment Failed	✗	✓	✗	✗
Display Out of Stock	✗	✗	✓	✓
Cancel Order	✗	✓	✓	✓

3. Entity-Relationship (E-R) Diagram



Relationship Summary

- One **Customer** can place multiple **Orders** (1:M).
- One **Order** can contain multiple **Products**.
- One **Product** can appear in multiple **Orders** (M:N relationship), which is resolved using the **ORDER_ITEM** entity.
- **ORDER_ITEM** stores the quantity of each product in an order.

Conclusion

The **DFDs** illustrate the flow of order and payment data through the system, the **Decision Table** defines the rules for order approval and payment processing, and the **E-R Diagram** models the relationships among customers, orders, products, and order items. These models provide a complete view of the Online Shopping System's processing logic and database structure.

Practical No.8: Draw class diagram and Sequence diagram, State Transition Diagram.

Conclusion : Successfully drew class diagram and Sequence diagram, State Transition Diagram.

Practical related questions

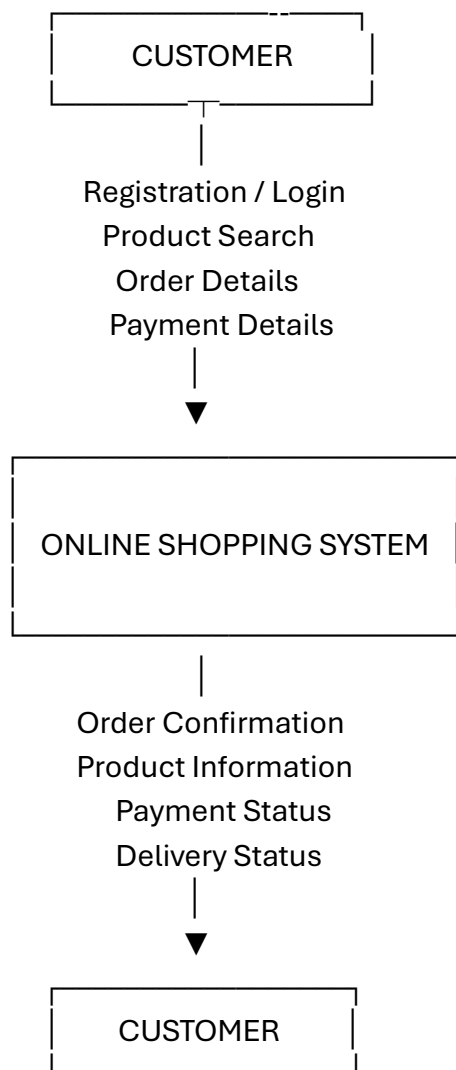
1. Create detailed DFDs, decision tables, and E-R diagrams for an Online Shopping System to show data flow, decision points, and entity relationships.

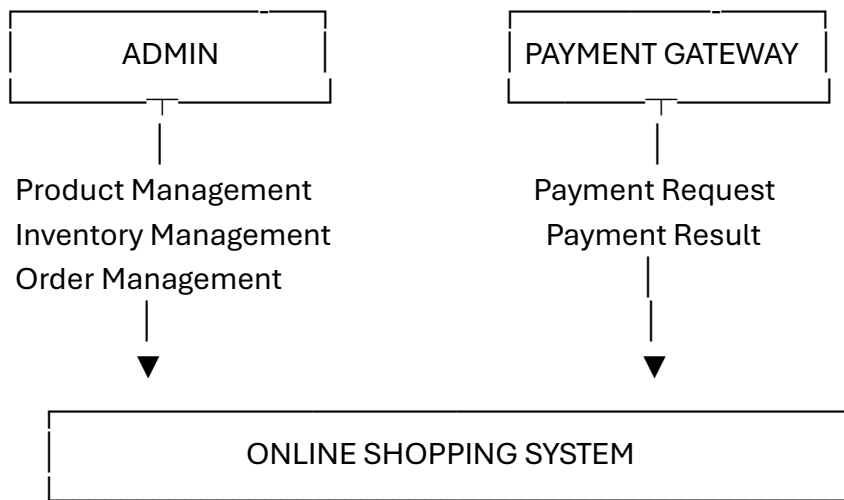
Answer :

A. Data Flow Diagrams (DFDs)

Level 0 DFD – Context Diagram

The **Level 0 DFD** represents the Online Shopping System as a single process and shows its interaction with external entities.



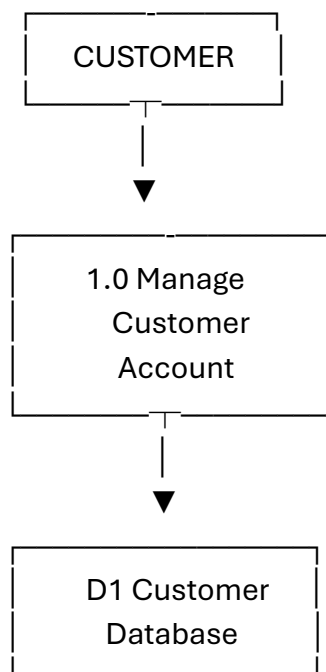


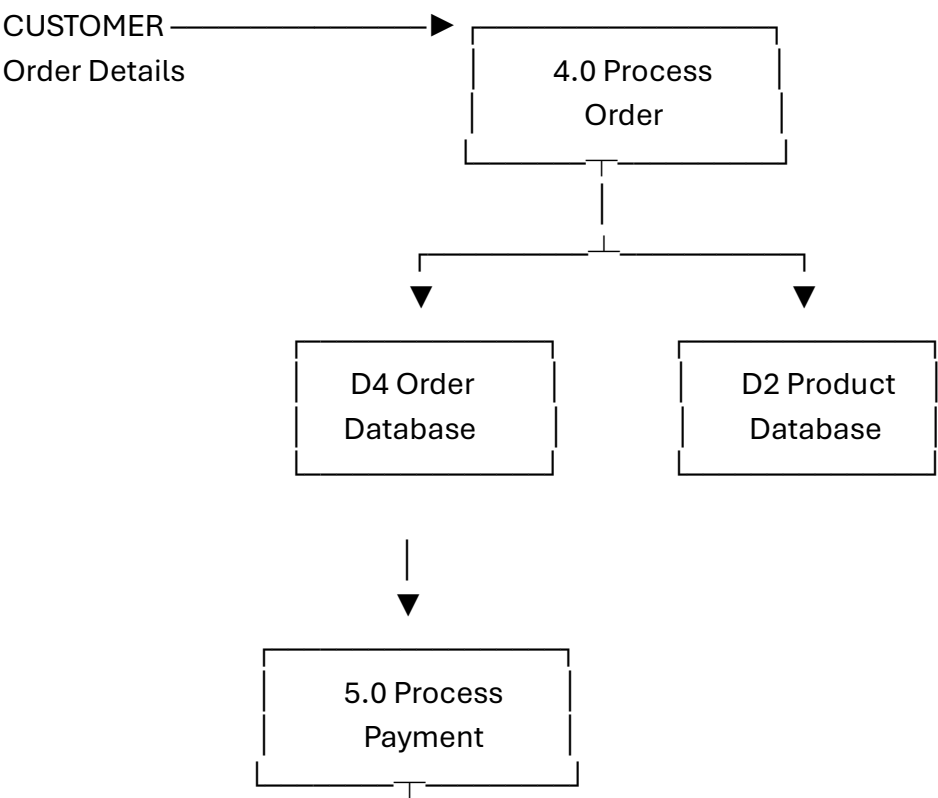
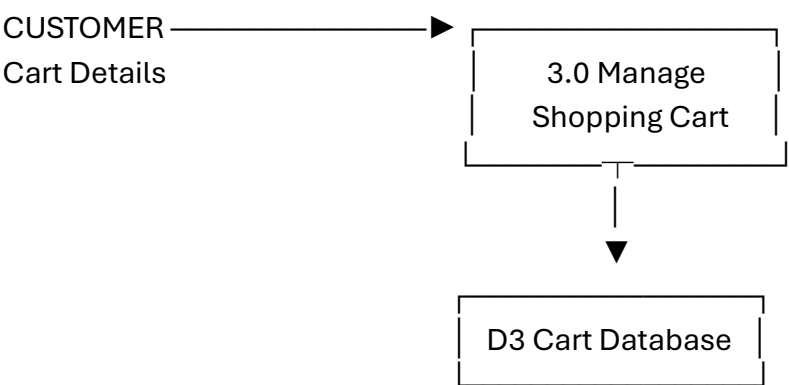
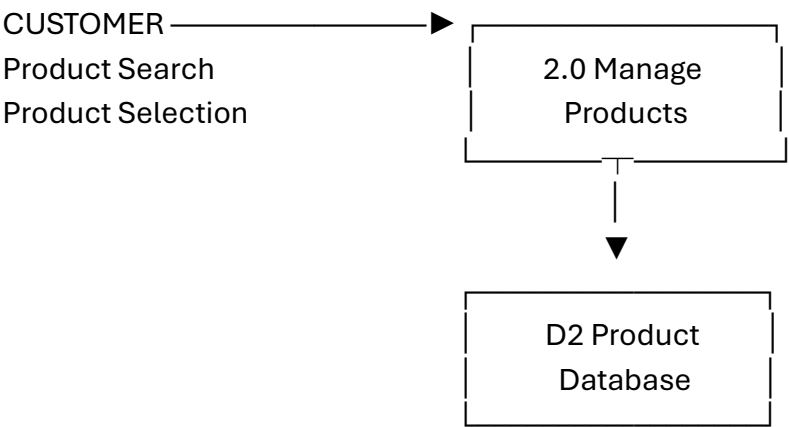
External Entities

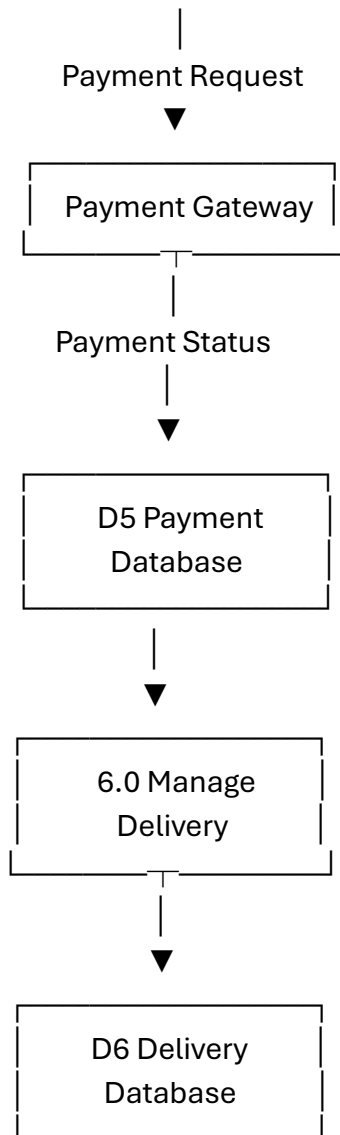
1. **Customer** – searches products, places orders, makes payments and receives order information.
2. **Admin** – manages products, inventory, customers and orders.
3. **Payment Gateway** – processes online payments and returns payment status.

Level 1 DFD – Online Shopping System

The system can be divided into several major processes.



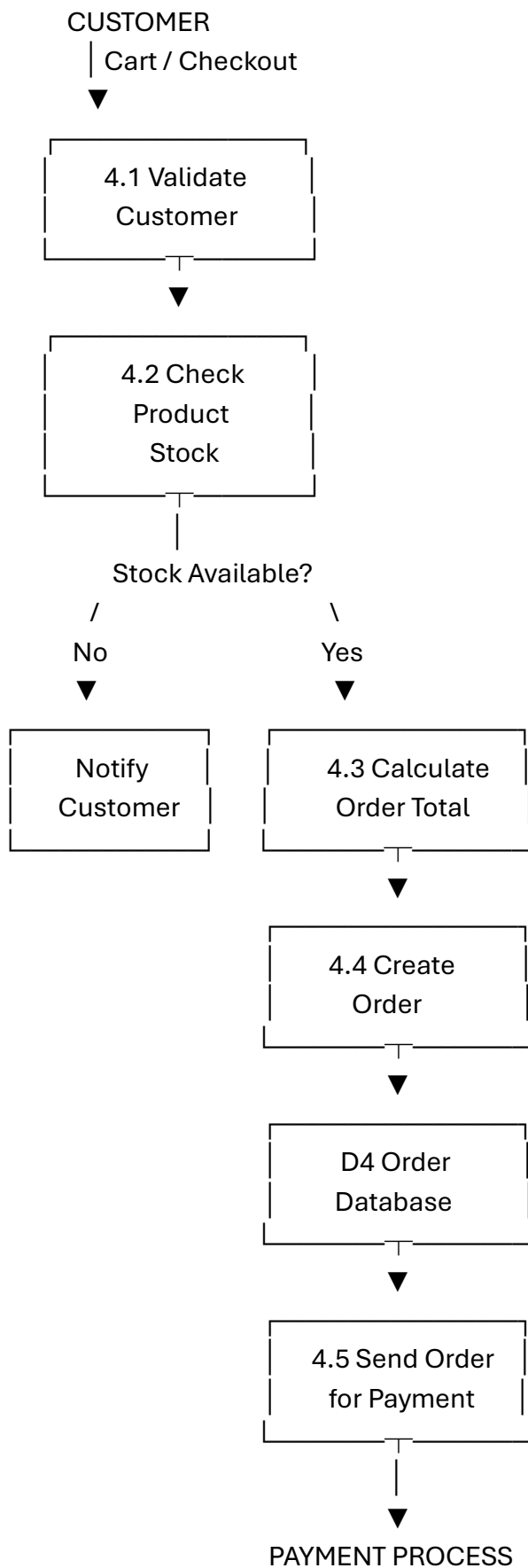




Major Processes

Process	Description
1.0 Manage Customer Account	Handles registration, login and customer details
2.0 Manage Products	Displays product information and handles product searches
3.0 Manage Shopping Cart	Adds, removes and updates products in the cart
4.0 Process Order	Creates and stores customer orders
5.0 Process Payment	Sends payment information to the payment gateway
6.0 Manage Delivery	Tracks and updates delivery status

Level 2 DFD – Order Processing



B. Decision Tables

Decision tables are used to represent **decision points and their possible outcomes** in the Online Shopping System.

Decision Table 1 – Order Processing

Conditions:

- Customer logged in?
- Product available?
- Payment successful?

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4
Customer logged in?	No	Yes	Yes	Yes
Product available?	-	No	Yes	Yes
Payment successful?	-	-	No	Yes
Allow checkout	No	No	Yes	Yes
Create order	No	No	No	Yes
Display login message	Yes	No	No	No
Display out-of-stock message	No	Yes	No	No
Display payment failure	No	No	Yes	No
Confirm order	No	No	No	Yes

Explanation

- **Rule 1:** If the customer is not logged in, checkout is not allowed.
- **Rule 2:** If the customer is logged in but the product is unavailable, the order cannot be created.
- **Rule 3:** If the product is available but payment fails, the order is not confirmed.
- **Rule 4:** If the customer is logged in, the product is available and payment succeeds, the order is confirmed.

Decision Table 2 – Payment Approval

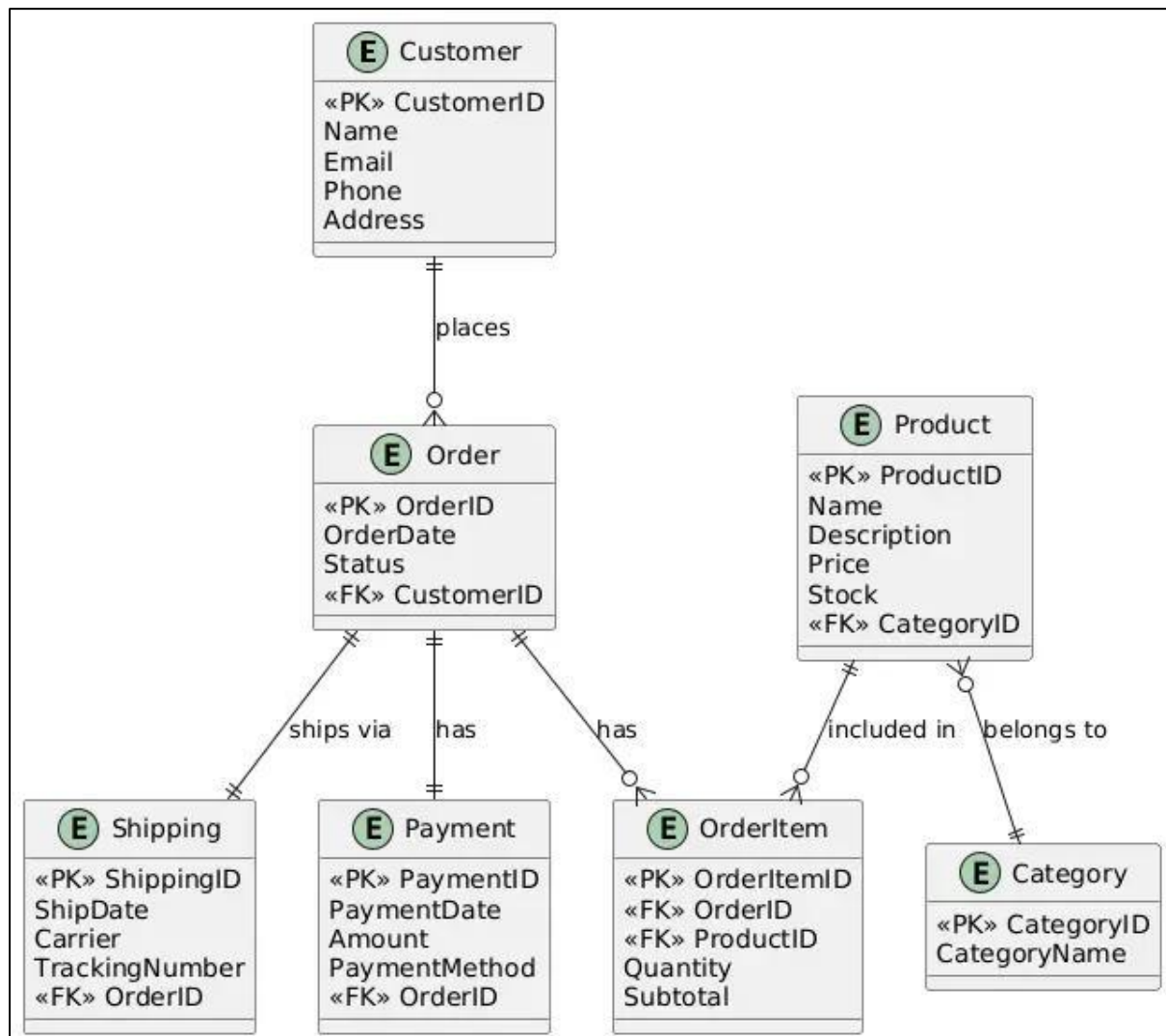
Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4
Valid payment details?	No	Yes	Yes	Yes
Sufficient funds/credit?	-	No	Yes	Yes
Payment gateway available?	-	-	No	Yes
Approve payment	No	No	No	Yes
Reject payment	Yes	Yes	Yes	No
Generate payment confirmation	No	No	No	Yes
Notify customer	Yes	Yes	Yes	No

Decision Table 3 – Product Availability

Conditions / Actions	Rule 1	Rule 2	Rule 3
Product exists?	No	Yes	Yes
Stock quantity > 0?	-	No	Yes
Product active?	-	-	Yes
Display product	No	Yes	Yes
Allow purchase	No	No	Yes
Show out-of-stock message	No	Yes	No
Show unavailable message	Yes	No	No

C. E-R Diagram for Online Shopping System

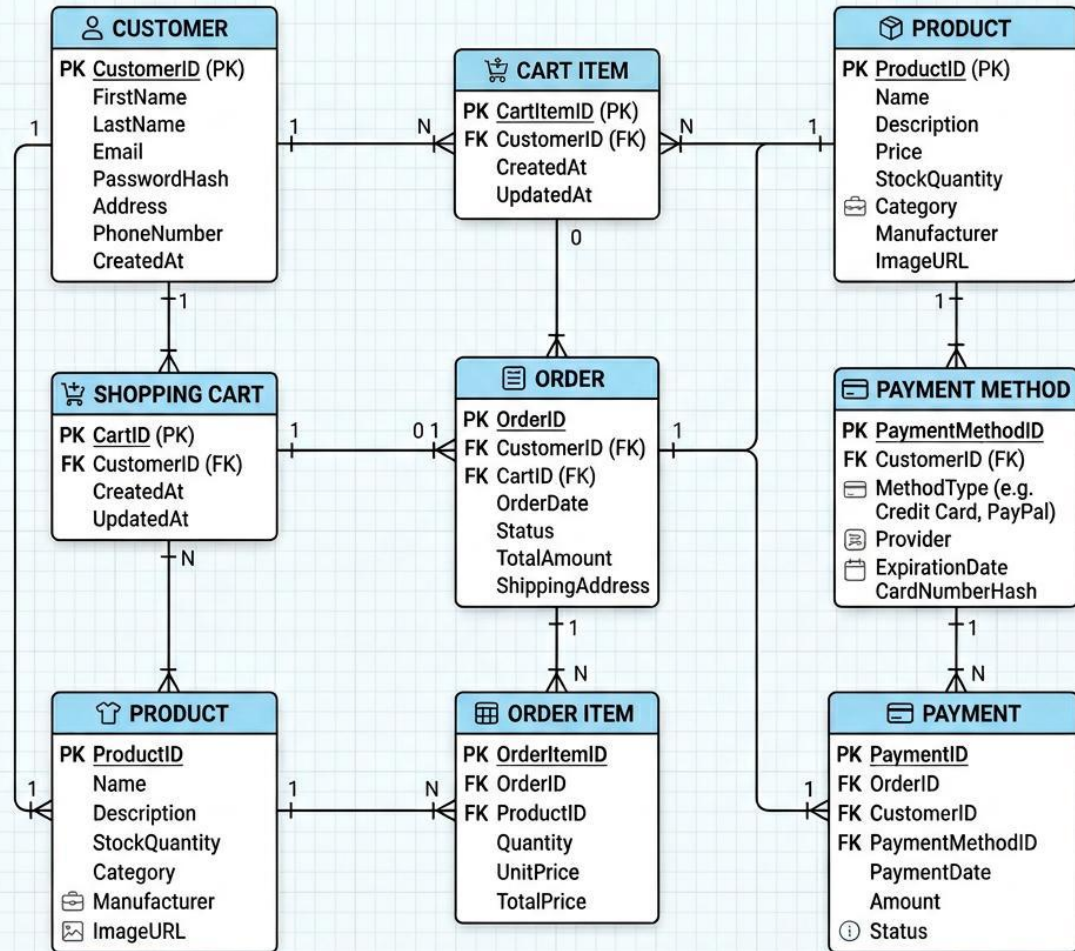
An **Entity-Relationship (E-R) diagram** represents entities, their attributes, and relationships within the system.



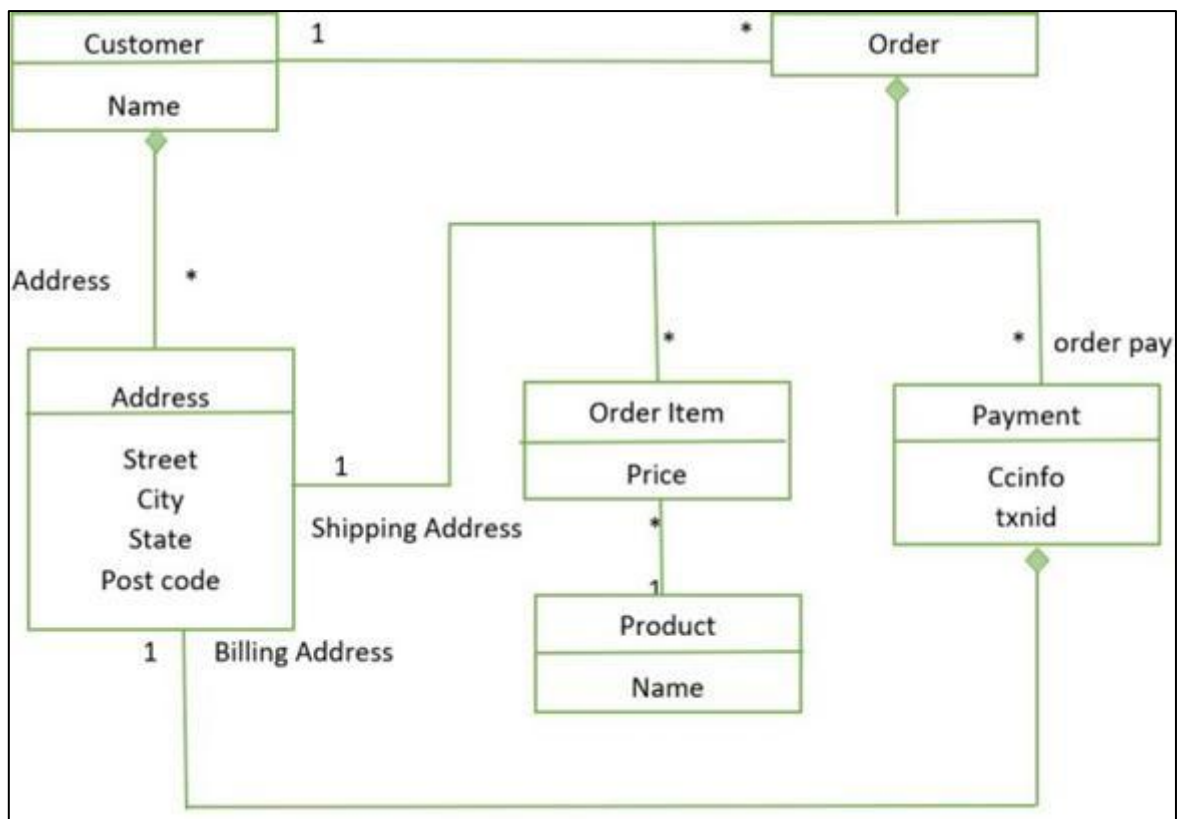
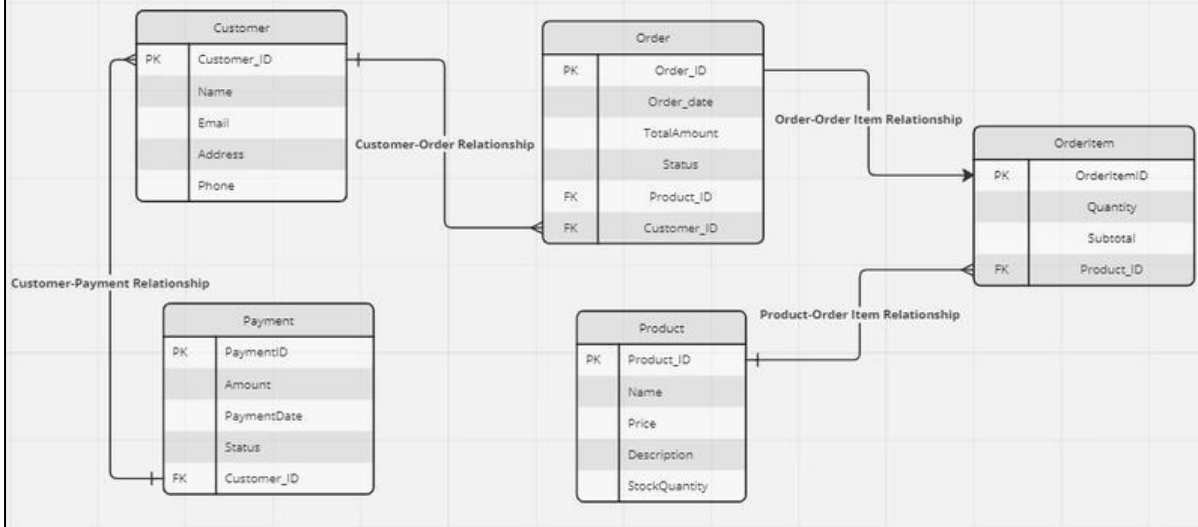
ONLINE SHOPPING SYSTEM ERD

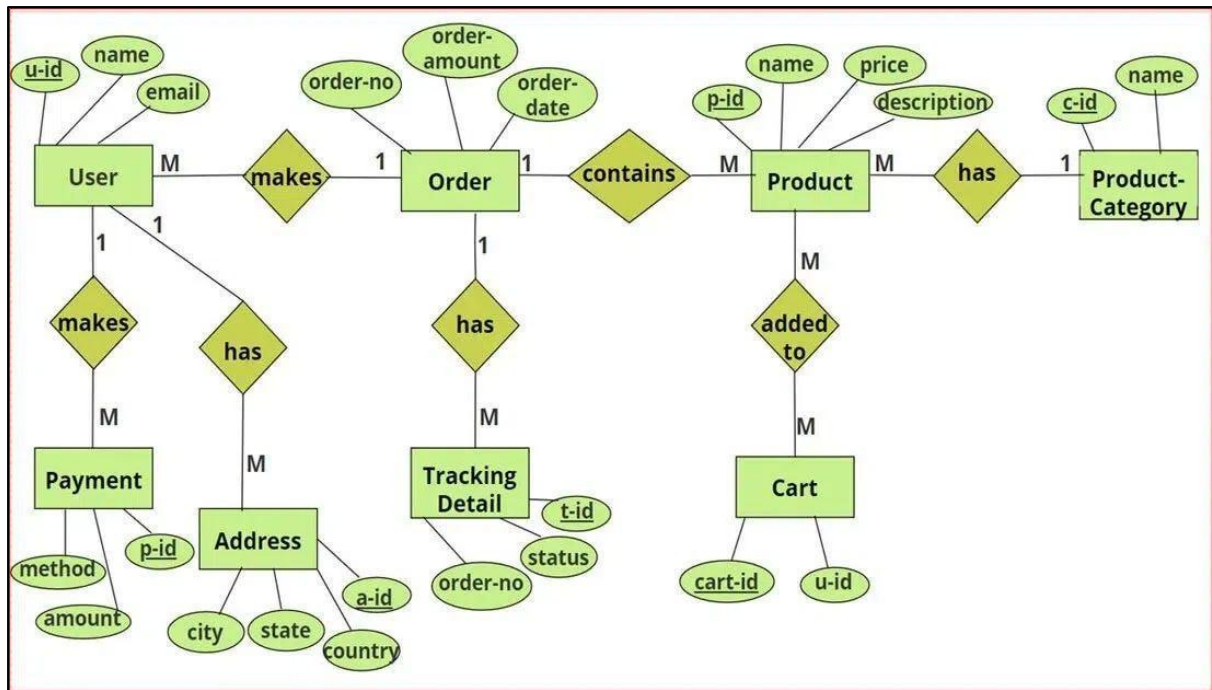
(Entity-Relationship Diagram)

1 ——— Mandatory One
 ——— Zero or Many
 ——— Mandatory Many



ER Diagram Sample for Ecommerce Project





Main Entities

1. CUSTOMER

Attributes:

- Customer_ID (**Primary Key**)
- Name
- Email
- Phone
- Address
- Password

2. PRODUCT

Attributes:

- Product_ID (**Primary Key**)
- Product_Name
- Description
- Price
- Quantity
- Category_ID (**Foreign Key**)

3. CATEGORY

Attributes:

- Category_ID (**Primary Key**)
- Category_Name
- Description

4. ORDER

Attributes:

- Order_ID (**Primary Key**)
- Order_Date
- Customer_ID (**Foreign Key**)
- Total_Amount
- Order_Status

5. ORDER_ITEM

Attributes:

- Order_Item_ID (**Primary Key**)
- Order_ID (**Foreign Key**)
- Product_ID (**Foreign Key**)
- Quantity
- Unit_Price
- Subtotal

6. PAYMENT

Attributes:

- Payment_ID (**Primary Key**)
- Order_ID (**Foreign Key**)
- Payment_Date
- Payment_Method
- Amount
- Payment_Status
- Transaction_ID

7. DELIVERY

Attributes:

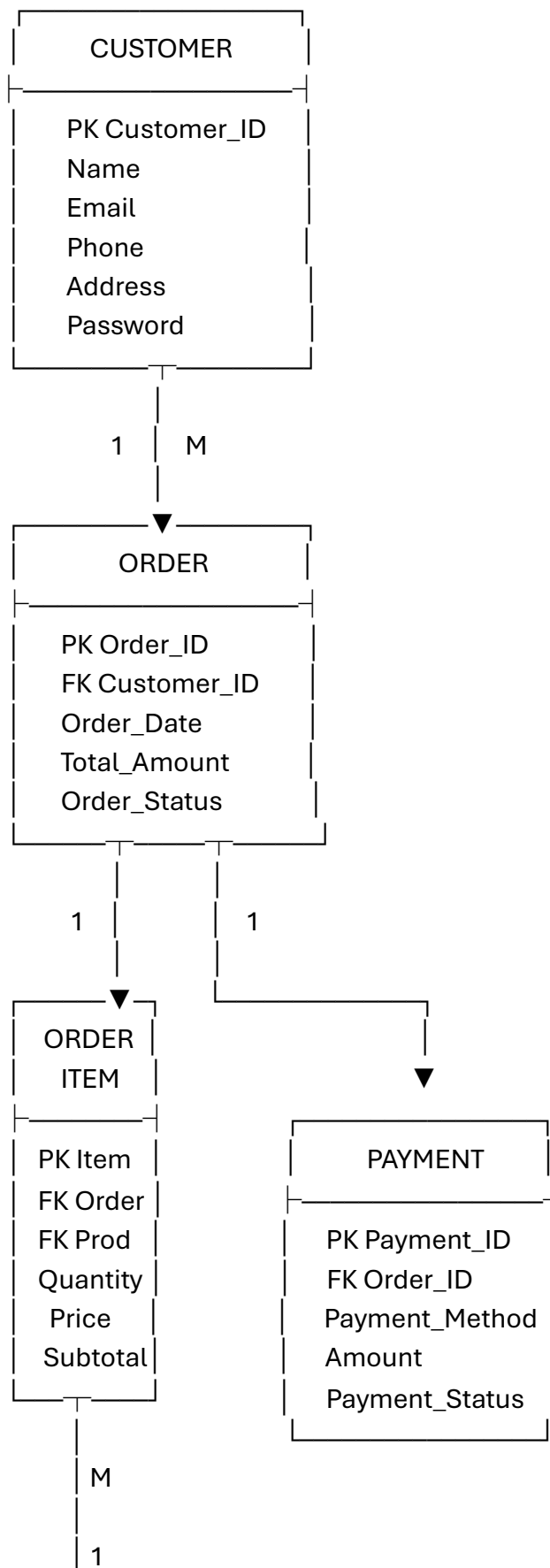
- Delivery_ID (**Primary Key**)
- Order_ID (**Foreign Key**)
- Delivery_Address
- Delivery_Date
- Delivery_Status
- Tracking_Number

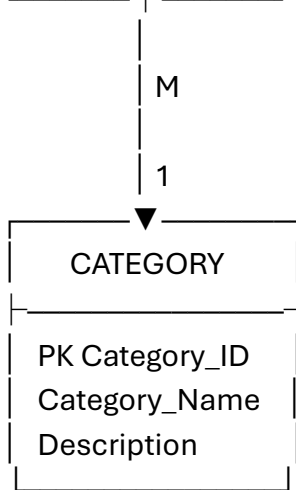
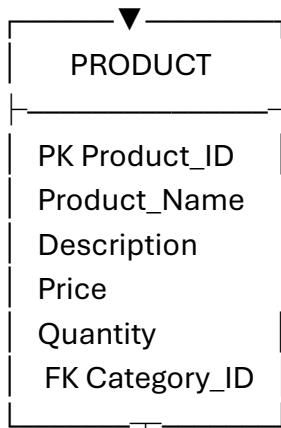
8. ADMIN

Attributes:

- Admin_ID (**Primary Key**)
- Name
- Email
- Password

E-R Diagram Structure





M
1

D. Relationships and Cardinality

Relationship	Cardinality	Description
Customer → Order	1 : M	One customer can place many orders
Order → Order_Item	1 : M	One order can contain multiple items
Product → Order_Item	1 : M	One product can appear in many order items
Category → Product	1 : M	One category can contain many products
Order → Payment	1 : 1	Each order has one payment record
Order → Delivery	1 : 1	Each order has one delivery record
Admin → Product	1 : M	An admin can manage multiple products
Admin → Order	1 : M	An admin can manage multiple orders

Important Point

The relationship between **ORDER** and **PRODUCT** is logically **many-to-many (M:N)** because:

- One order can contain many products.
- One product can be included in many different orders.

This M:N relationship is resolved using the **ORDER_ITEM** entity.

E. Overall System Flow

Customer



Register / Login



Browse Products



Select Product



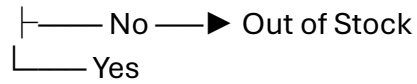
Add to Cart



Checkout



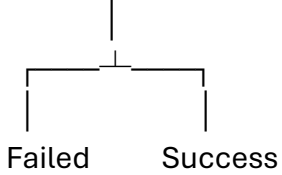
Check Product Availability



Calculate Total



Make Payment



Failed

Success



Payment
Failure



Create
Order



Update Inventory



Generate Delivery



Update Delivery Status



Order Confirmation



Customer

Conclusion

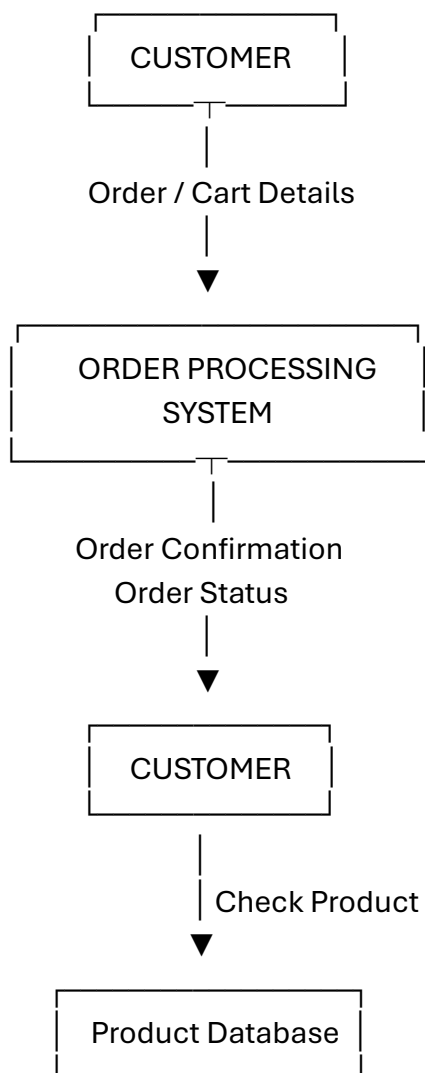
The **DFDs** show how data moves between customers, administrators, databases, payment gateways, and different system processes. The **decision tables** define the conditions under which orders, payments, and product purchases are accepted or rejected. The **E-R diagram** defines the database structure and relationships among **Customer, Product, Category, Order, Order Item, Payment, Delivery, and Admin** entities. Together, these models provide a complete logical representation of an Online Shopping System.

2. Develop DFDs and decision tables illustrating the order processing and payment approval steps, and draw an E-R diagram for customers, products, and orders.

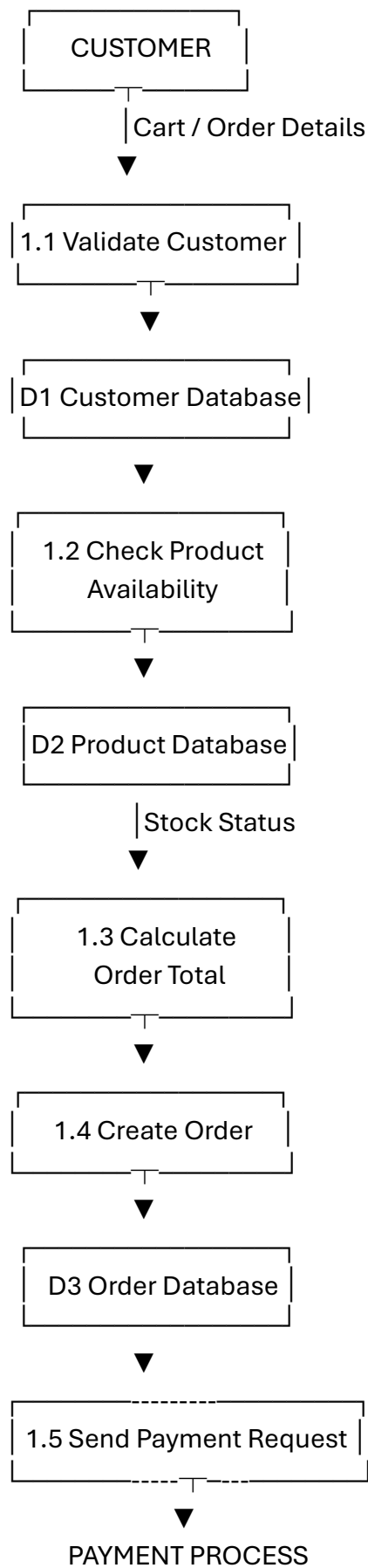
Answer :

A. DFD for Order Processing

Level 0 – Order Processing Context



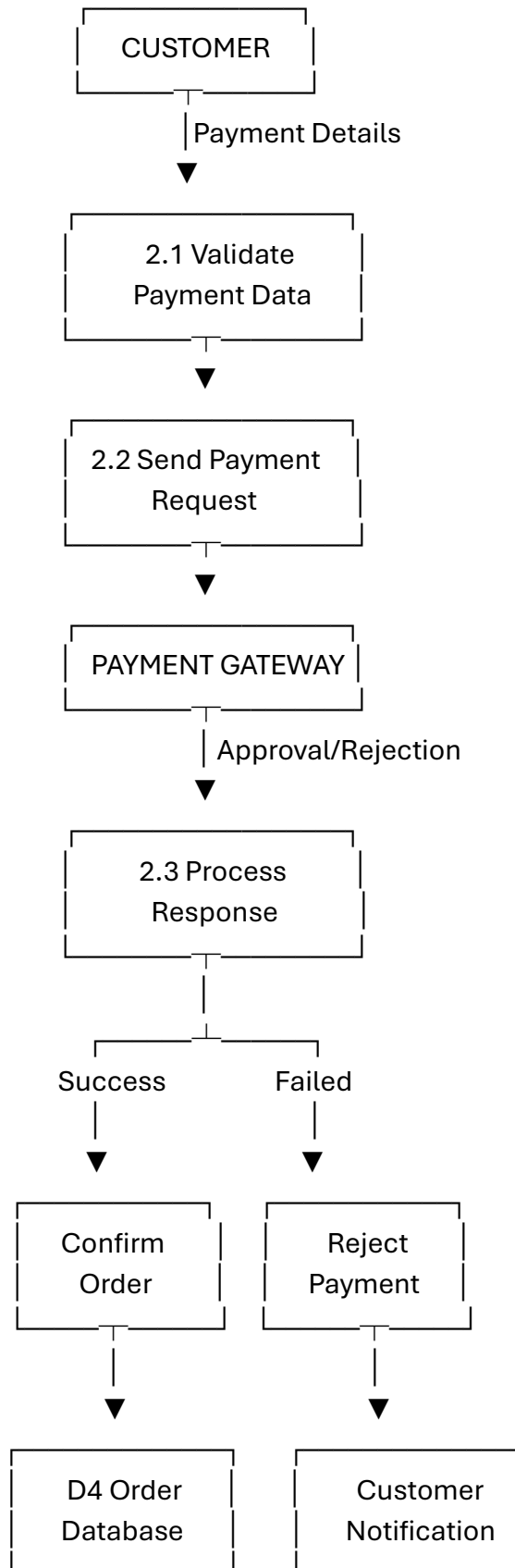
B. Level 1 DFD – Detailed Order Processing



Order Processing Flow

1. Customer selects products and proceeds to checkout.
2. The system validates the customer's account.
3. Product availability and stock quantity are checked.
4. The system calculates the total order amount.
5. An order is created and stored.
6. Payment details are sent for payment approval.
7. After successful payment, the order is confirmed.

C. DFD for Payment Approval
Level 1 – Payment Processing



D. Decision Table – Order Processing

The decision table determines whether an order can be successfully processed.

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4
Customer logged in?	No	Yes	Yes	Yes
Product available?	-	No	Yes	Yes
Valid delivery address?	-	-	No	Yes
Process order?	No	No	No	Yes
Ask customer to login	Yes	No	No	No
Show out-of-stock message	No	Yes	No	No
Request valid address	No	No	Yes	No
Create order	No	No	No	Yes

Interpretation

- **Rule 1:** Customer must log in before ordering.
- **Rule 2:** Order cannot be processed if the product is out of stock.
- **Rule 3:** A valid delivery address is required.
- **Rule 4:** If all conditions are satisfied, the order is created.

E. Decision Table – Payment Approval

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4
Payment details valid?	No	Yes	Yes	Yes
Sufficient funds/credit?	-	No	Yes	Yes
Payment gateway available?	-	-	No	Yes
Approve payment	No	No	No	Yes
Reject payment	Yes	Yes	Yes	No
Confirm order	No	No	No	Yes
Notify customer of failure	Yes	Yes	Yes	No

Interpretation

Rule 4 represents a successful payment because:

Valid payment details + sufficient funds + available payment gateway = **Payment Approved**

If any required condition fails, the payment is rejected and the customer is notified.

F. E-R Diagram – Customer, Product and Order

The main entities are:

CUSTOMER

- **Customer_ID (PK)**
- Name
- Email
- Phone
- Address
- Password

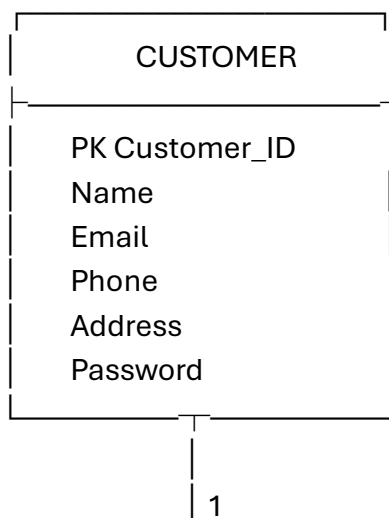
PRODUCT

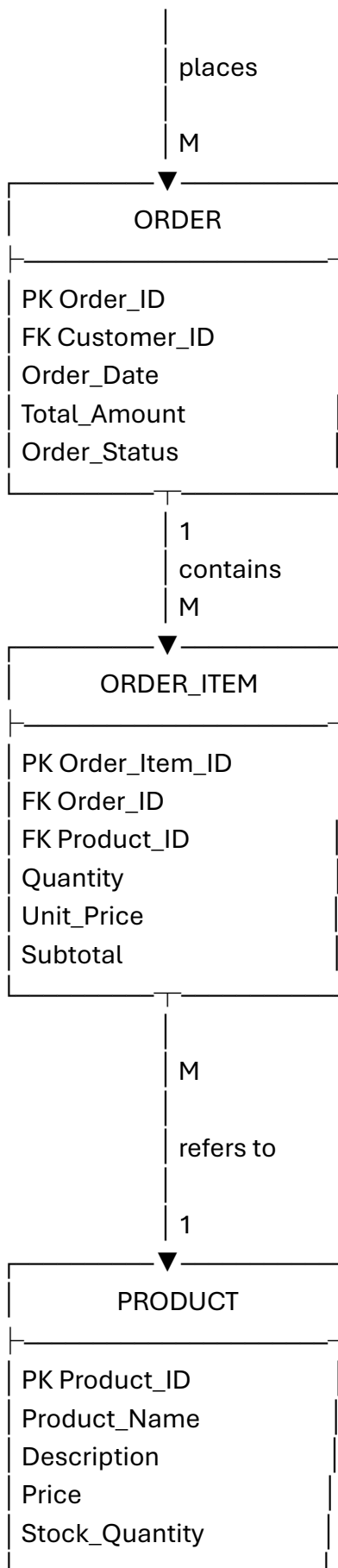
- **Product_ID (PK)**
- Product_Name
- Description
- Price
- Stock_Quantity

ORDER

- **Order_ID (PK)**
- Order_Date
- Customer_ID (FK)
- Total_Amount
- Order_Status
- **Order_Item_ID (PK)**
- Order_ID (FK)
- Product_ID (FK)
- Quantity
- Unit_Price
- Subtotal

E-R Diagram





G. Relationships

Relationship	Cardinality	Meaning
Customer → Order	1 : M	One customer can place many orders
Order → Order_Item	1 : M	One order can contain many items
Product → Order_Item	1 : M	One product can occur in many order items
Order ↔ Product	M : N	An order can contain many products and a product can belong to many orders

The **ORDER_ITEM** entity resolves the **many-to-many relationship** between Order and Product.

Overall Process

Customer



Select Products



Add to Cart



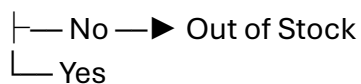
Checkout



Validate Customer



Check Stock



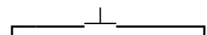
Create Order



Send Payment Request



Payment Gateway



Success

Failure



Confirm
Order

Reject
Payment



Update
Order
Status

Notify
Customer

This gives the required **DFDs for order processing and payment approval, decision tables for both decision-making stages, and an E-R diagram covering customers, products, orders, and order items.**

3. Differentiate between attributes and operations in a class diagram.

Answer :

Attribute	Operation
Represents the data or properties of a class.	Represents the behaviour or functionality of a class.
Describes what an object has .	Describes what an object can do .
Stores information about an object.	Performs an action or operation on the object/data.
Usually has a name and data type .	Usually has a name, parameters, and return type .
Written in the attribute section of a class.	Written in the operation/method section of a class.
Example: name : String	Example: login() : void
Example: price : float	Example: calculateTotal() : float

4. What is the purpose of visibility symbols (+, -, #) in a class diagram?

Answer :

Visibility symbols in a **UML class diagram** specify the **access level** of attributes and operations. They indicate which parts of a class can access a particular attribute or operation.

1. Public (+)

A **public** attribute or operation can be accessed from outside the class.

Example:

+ displayProduct()

+ productName : String

displayProduct() can be called by other classes.

2. Private (-)

A **private** attribute or operation can only be accessed within the class where it is declared.

Example:

- password : String

- calculatePrice()

This is commonly used for **data hiding and encapsulation**.

3. Protected (#)

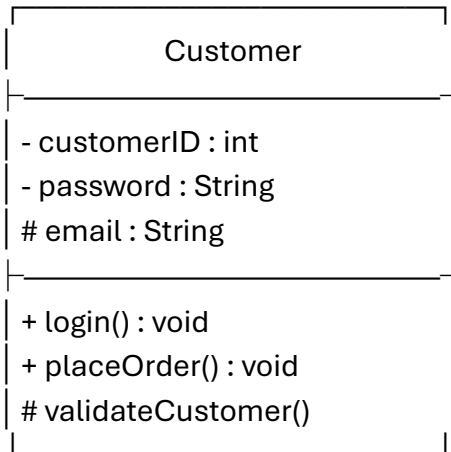
A **protected** attribute or operation can be accessed within the class and by its **subclasses (derived classes)**.

Example:

```
# employeeID : int
```

```
# calculateSalary()
```

Example Class Diagram



Practical No. 9: Create decision table for a project.

Conclusion : Successfully created decision table for a project.

Practical related questions

1. What are the main components of a decision table?

Answer :

Component	Description
1. Condition Stub	Lists all the conditions or factors that affect the decision.
2. Condition Entries	Shows the possible values of each condition, such as Yes/No , True/False , or specific values.
3. Action Stub	Lists the actions that the system can perform.
4. Action Entries	Shows which actions should be performed for each combination of conditions.
5. Rules	Each column represents a rule containing a specific combination of conditions and the corresponding actions.

2. What are the advantages of decision table?

Answer :

- 1. Easy to Understand**
Decision tables present conditions and actions in a clear and organized format.
- 2. Handles Complex Logic**
They can represent multiple conditions and their possible combinations effectively.
- 3. Reduces Errors**
Decision tables help identify missing, conflicting, or incorrect rules.
- 4. Ensures Completeness**
They help ensure that all possible combinations of conditions have been considered.
- 5. Easy to Modify**
Rules can be added, removed, or changed without redesigning the entire logic.
- 6. Improves Communication**
They provide a common format that can be easily understood by developers, analysts, testers, and users.
- 7. Useful for Testing**
Each rule can be converted into a **test case**, making it easier to verify system behaviour.

8. Avoids Ambiguity

Decision tables clearly show which action should be performed for each combination of conditions.

3. Create a decision table for an Online Shopping System.

Answer :

A decision table for an **Online Shopping System** can be used to determine whether an order should be successfully placed based on customer login, product availability, delivery details, and payment status.

Decision Table

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5
Customer logged in?	No	Yes	Yes	Yes	Yes
Product available?	-	No	Yes	Yes	Yes
Valid delivery address?	-	-	No	Yes	Yes
Payment successful?	-	-	-	No	Yes
Allow checkout	No	No	No	Yes	Yes
Display login message	Yes	No	No	No	No
Display out-of-stock message	No	Yes	No	No	No
Request valid address	No	No	Yes	No	No
Display payment failure	No	No	No	Yes	No
Create/confirm order	No	No	No	No	Yes

Explanation of Rules

- **Rule 1:** If the customer is not logged in, the system asks the customer to log in.
- **Rule 2:** If the customer is logged in but the product is unavailable, the system displays an **out-of-stock** message.
- **Rule 3:** If the product is available but the delivery address is invalid, the system asks for a valid address.
- **Rule 4:** If all order details are valid but payment fails, the system displays a **payment failure** message.
- **Rule 5:** If the customer is logged in, the product is available, the address is valid, and payment is successful, the **order is confirmed**.

Conclusion

This decision table ensures that all important conditions are considered before an order is confirmed. It helps make the **order-processing logic clear, complete, and easy to test**.

Practical No.10: Write test cases to validate requirements from SRS document.

Conclusion : Successfully wrote test cases to validate requirements from SRS document.

Practical related questions

1. Identify functional and non-functional requirements from a given SRS of an Online Library Management System.

Answer :

A. Functional Requirements

Functional requirements describe the **specific functions or services** that the system must provide.

No.	Functional Requirement
1	User Registration: Users should be able to create a library account.
2	User Login: Registered users should be able to log in securely.
3	Book Search: Users should be able to search for books by title, author, category, or ISBN.
4	Book Availability: The system should display whether a book is available, issued, or reserved.
5	Book Issue: Librarians should be able to issue available books to registered users.
6	Book Return: The system should record returned books and update their availability.
7	Book Reservation: Users should be able to reserve books that are currently unavailable.
8	Fine Calculation: The system should automatically calculate fines for overdue books.
9	Book Management: The librarian/admin should be able to add, update, and remove book records.
10	User Management: The administrator should be able to add, update, or deactivate user accounts.
11	Due-Date Management: The system should record and display the due date for issued books.
12	Notifications: The system should notify users about due dates, overdue books, reservations, and fines.
13	Reports: The system should generate reports such as issued books, returned books, overdue books, and collected fines.
14	Logout: Users should be able to securely log out of their accounts.

B. Non-Functional Requirements

Non-functional requirements describe **how well the system should perform** and the constraints on its operation.

No.	Non-Functional Requirement
1	Performance: Search results should be displayed within a reasonable response time.
2	Security: User passwords and personal information must be protected from unauthorized access.
3	Reliability: The system should operate correctly with minimal failures.
4	Availability: The library system should be available whenever users need to access it.
5	Usability: The interface should be simple and easy for students, librarians, and administrators to use.
6	Scalability: The system should support an increasing number of users and books without significant performance degradation.
7	Maintainability: The software should be easy to update, maintain, and modify.
8	Backup and Recovery: Library data should be backed up regularly and recoverable after system failure.
9	Compatibility: The system should work properly on commonly used web browsers and devices.
10	Data Integrity: Book, user, issue, return, and fine records should remain accurate and consistent.

2. Write detailed test cases to validate each requirement from the SRS, including scenario, input, and expected output.

Answer :

A. Functional Requirement Test Cases

TC ID	Requirement / Scenario	Input	Expected Output
TC01	User Registration with valid details	Name, valid email, password, phone number	Account is created successfully and a confirmation message is displayed.
TC02	User Registration with invalid email	Name, abc@, password	System rejects the registration and displays an invalid email message.
TC03	User Login with valid credentials	Registered email and correct password	User is successfully logged in and redirected to the dashboard.

TC ID	Requirement / Scenario	Input	Expected Output
TC04	User Login with incorrect credentials	Registered email and incorrect password	Login is rejected and an error message is displayed.
TC05	Search book by title	Enter Database Management System	Matching books are displayed.
TC06	Search book by author	Enter an author's name	Books written by that author are displayed.
TC07	Search for unavailable book	Search for a book currently issued	Book is displayed with "Not Available" status.
TC08	Check book availability	Select an available book	System displays available quantity/status.
TC09	Issue an available book	Valid user + available book	Book is issued and issue record with due date is created.
TC10	Issue an unavailable book	Valid user + unavailable book	System prevents issuing and displays an unavailable message.
TC11	Return an issued book	Book ID + valid issue record	Book is marked as returned and availability is updated.
TC12	Return overdue book	Book returned after due date	Book is returned and applicable fine is calculated.
TC13	Reserve unavailable book	Valid user + currently issued book	Reservation is created successfully.
TC14	Reserve available book	Valid user + available book	System informs the user that the book is available instead of creating an unnecessary reservation.
TC15	Calculate overdue fine	Book returned after the due date	System calculates and displays the correct fine.
TC16	Add new book	Valid book details entered by librarian	New book is added to the library database.
TC17	Update book details	Change book price/category/details	Updated information is saved successfully.
TC18	Delete book	Select an existing book and choose delete	Book is removed or marked inactive after confirmation.
TC19	Add new user	Valid user information entered by admin	New user account is created.
TC20	Deactivate user	Select an active user and deactivate	User account is deactivated and login is prevented.

TC ID	Requirement / Scenario	Input	Expected Output
TC21	Display due date	User opens issued-book details	Correct due date is displayed.
TC22	Due-date notification	Book approaches its due date	System sends/displays a due-date reminder.
TC23	Overdue notification	Book passes its due date	System sends/displays an overdue notification.
TC24	Generate issued-books report	Librarian selects "Issued Books Report"	System generates a report containing currently issued books.
TC25	Generate overdue-books report	Librarian selects "Overdue Books Report"	System displays all currently overdue books.
TC26	User Logout	Click Logout	User is logged out and the session is terminated.

B. Non-Functional Requirement Test Cases

TC ID	Requirement / Scenario	Input / Test Condition	Expected Output
TC27	Performance	Search for a commonly used book	Search results should appear within the specified response-time limit.
TC28	Security – Password Protection	Enter password during registration	Password should not be displayed or stored as plain text.
TC29	Security – Unauthorized Access	Try accessing admin functions as a normal user	Access should be denied.
TC30	Reliability	Perform repeated book searches and transactions	System should continue functioning without errors or crashes.
TC31	Availability	Access the system during normal operating hours	System should be accessible and operational.
TC32	Usability	New user performs book search	User should be able to search without requiring extensive assistance.
TC33	Scalability	Simulate a large number of users/books	System should continue operating with acceptable performance.

TC ID	Requirement / Scenario	Input / Test Condition	Expected Output
TC34	Maintainability	Update a book-management module	Changes should not unexpectedly affect unrelated modules.
TC35	Backup and Recovery	Restore the system after simulated data loss	Previously backed-up library data should be recovered correctly.
TC36	Compatibility	Open the system using supported browsers/devices	System should function correctly across supported environments.
TC37	Data Integrity	Issue and return a book	Book status, user record, and transaction record should remain consistent.

3. Explain the importance of validating SRS requirements through test cases in ensuring software quality.

Answer :

SRS (Software Requirements Specification) defines what a software system should do and the constraints it should follow. Validating SRS requirements through **test cases** ensures that the developed software actually satisfies those requirements.

Importance of Test Cases in SRS Validation

1. **Ensures Requirements Are Met**
Test cases verify that every requirement specified in the SRS has been correctly implemented.
2. **Detects Errors Early**
Testing can identify defects before the software is delivered to users, reducing the cost and effort of fixing them later.
3. **Improves Software Quality**
Systematic testing ensures that the software is reliable, accurate, secure, and performs its intended functions.
4. **Ensures Complete Requirement Coverage**
Each SRS requirement can be mapped to one or more test cases. This helps ensure that no important requirement is overlooked.
5. **Validates Expected Behaviour**
Test cases define the expected output for a given input. The actual result can then be compared with the expected result.
6. **Reduces Ambiguity**
Writing test cases can reveal requirements that are unclear, incomplete, inconsistent, or difficult to test.

7. Improves Reliability and User Satisfaction

Proper validation ensures that the system behaves correctly under different conditions, resulting in a better user experience.

8. Supports Regression Testing

After modifications or updates, existing test cases can be executed again to ensure that previously working functionality has not been broken.

9. Provides Documentation

Test cases serve as documented evidence that the software requirements have been tested and validated.

10. Helps in Acceptance Testing

Successful execution of requirement-based test cases provides confidence that the software is ready for delivery and meets customer expectations.

Practical No.11: Prepare test cases for Black Box Testing

Conclusion : Successfully prepared test cases for Black Box Testing

Practical related questions

1. What is a Black box Testing?

Answer :

Black Box Testing is a software testing technique in which the tester checks the **functionality of a software system without knowing or examining its internal source code or implementation details.**

The tester provides **inputs** to the system and checks whether the **outputs match the expected results.**

Example

For an **Online Library Management System**, consider the login feature:

Input	Expected Output
Correct username + correct password	Login successful
Correct username + wrong password	Login failed
Empty username/password	Validation message displayed

The tester does not need to know how the login code works. They only verify the system's behaviour.

2. What is Test Case?

Answer :

A **Test Case** is a set of **conditions, inputs, steps, and expected results** used to verify whether a particular software function works correctly according to its requirements.

A test case helps the tester determine whether the **actual output** of the system matches the **expected output.**

3. What is Equivalence Partitioning? Give an example.

Answer :

Equivalence Partitioning (EP) is a **Black Box Testing technique** in which the input data is divided into groups or **partitions** where all values in a group are expected to behave similarly.

Instead of testing every possible input, the tester selects **one or more representative values from each partition.**

Example

Suppose an **Online Library Management System** allows a user to borrow **1 to 5 books** at a time.

The input can be divided into three equivalence partitions:

Partition	Input Range	Type	Example Test Value
Invalid	Less than 1	Invalid	0
Valid	1–5	Valid	3
Invalid	Greater than 5	Invalid	6

4. How would you test a login page using Black Box Testing?

Answer :

Test Cases

Test Case	Input	Expected Output
TC01 – Valid Login	Correct username + correct password	User is successfully logged in and redirected to the dashboard.
TC02 – Wrong Password	Correct username + incorrect password	Error message such as " Invalid username or password " is displayed.
TC03 – Wrong Username	Incorrect username + correct password	Login is rejected and an error message is displayed.
TC04 – Both Incorrect	Incorrect username + incorrect password	Login is rejected.
TC05 – Empty Username	Blank username + valid password	System displays " Username is required ".
TC06 – Empty Password	Valid username + blank password	System displays " Password is required ".
TC07 – Both Fields Empty	Blank username + blank password	Required-field validation messages are displayed.
TC08 – Invalid Username Format	Invalid email/username format	System displays an appropriate validation message.
TC09 – Password Boundary	Password at minimum/maximum allowed length	System accepts or rejects it according to the specified password rules.
TC10 – Special Characters	Username/password containing special characters	System handles the input correctly according to the requirements.
TC11 – Multiple Failed Attempts	Several incorrect passwords	System applies the specified account-lockout or security mechanism, if required.
TC12 – Logout	Successfully logged-in user clicks Logout	User is logged out and cannot access protected pages without logging in again.

Black Box Testing Approach

The tester would:

1. Open the login page.
2. Enter different combinations of **valid and invalid inputs**.
3. Click the **Login** button.
4. Observe the system's response.
5. Compare the **actual output with the expected output**.
6. Mark each test case as **Pass or Fail**.

Example

Input:

Username = user@gmail.com

Password = wrong123

Expected Output:

"Invalid username or password"

Actual Output:

"Invalid username or password"

Result:

PASS

The tester does **not** need to know how the authentication algorithm, database query, or password verification works.

5. What will you do if the actual result differs from the expected result?

Answer :

If the **actual result differs from the expected result**, the test case is considered **failed**. The tester should investigate and report the issue.

Steps to Follow:

1. **Mark the Test Case as Failed**
Record the test result as **Fail**.
2. **Verify the Test Steps and Input**
Check whether the correct test data, preconditions, and test steps were used.
3. **Reproduce the Issue**
Execute the test again to confirm that the problem consistently occurs.
4. **Document the Defect**
Record details such as:

- Test Case ID
 - Steps to reproduce
 - Input data
 - Expected result
 - Actual result
 - Screenshots/logs, if available
5. **Report the Bug**
Submit the defect to the development team through the project's bug-tracking system.
 6. **Developer Fixes the Issue**
The developer investigates the cause and implements a correction.
 7. **Retest the Functionality**
Run the failed test case again after the fix.
 8. **Perform Regression Testing**
Check related features to ensure that the fix has not introduced new problems.
 9. **Update the Test Result**
If the expected result is achieved after the fix, mark the test case as **Pass**.

Example

Scenario: Login with valid credentials

- **Expected Result:** User should be redirected to the dashboard.
- **Actual Result:** An "Invalid password" message is displayed.

Action: Mark the test as **Failed**, verify the credentials, reproduce the issue, report the defect, and retest after the developer fixes it.

Practical No.12: Identify risks involved in the project and prepare RMMM (RMMM-Risk Management, Mitigation and Monitoring) plan.

Conclusion : Successfully identified risks involved in the project and prepare RMMM (RMMM-Risk Management, Mitigation and Monitoring) plan.

Practical related questions

1. What does RMMM stand for and why is it important?

Answer :

RMMM stands for **Risk Mitigation, Monitoring, and Management**.

It is a plan used in **software project management** to identify potential risks, reduce their impact, monitor them throughout the project, and take appropriate action if they occur.

Importance of RMMM :

- **Identifies risks early** before they become major problems.
- **Reduces project failures** by preparing responses to potential risks.
- **Minimizes cost and schedule overruns**.
- **Improves decision-making** by providing planned responses to risks.
- **Ensures continuous monitoring** of project risks.
- **Improves software quality** by addressing technical and operational risks.
- **Increases the chances of project success**.

2. What is the difference between risk mitigation and risk monitoring?

Answer :

Risk Mitigation	Risk Monitoring
It involves taking actions to reduce the probability or impact of a risk .	It involves continuously observing and tracking identified risks .
It is mainly preventive .	It is mainly observational and tracking-oriented .
It is performed to prevent a risk from becoming a serious problem.	It is performed to determine whether the risk is changing or becoming more likely.
Focuses on what can be done to reduce the risk .	Focuses on whether the risk is occurring or changing .
Example: Taking regular database backups to reduce the impact of database failure.	Example: Regularly checking database health and backup status.

3. What is the purpose of risk monitoring?

Answer :

1. **Identify Changes in Risks**
Determines whether the probability or impact of an existing risk has increased or decreased.
2. **Detect New Risks**
Helps identify risks that were not recognized during the initial risk assessment.
3. **Check Risk Mitigation Measures**
Ensures that the actions taken to reduce risks are working effectively.
4. **Enable Early Warning**
Helps the project team recognize warning signs before a risk becomes a major problem.
5. **Support Timely Decision-Making**
Provides information needed to take corrective or preventive actions at the right time.
6. **Track Risk Status**
Keeps records of whether risks are **active, reduced, resolved, or no longer relevant**.

4. How can you reduce the impact of a risk?

Answer :

1. **Identify Risks Early**
Identify possible risks at the beginning of the project so that suitable actions can be planned.
 2. **Create a Risk Mitigation Plan**
Define specific actions to reduce the probability or impact of each risk.
 3. **Use Backup and Recovery**
Keep regular backups of important data to reduce the impact of data loss or system failure.
 4. **Provide Training**
Train team members to reduce errors caused by lack of technical knowledge.
 5. **Use Redundancy**
Have backup servers, developers, or resources so that the project can continue if one resource becomes unavailable.
 6. **Perform Regular Testing**
Frequent testing helps identify and fix defects before they become serious problems.
 7. **Monitor Risks Continuously**
Track risks throughout the project to detect warning signs early.
 8. **Prepare Contingency Plans**
Have an alternative plan ready if the risk actually occurs.
-

Practical No.13: Calculate size of the project using Function point metric.

Conclusion : Successfully calculated size of the project using Function point metric.

Practical related questions

1. What is the purpose of using Function Point Analysis in software project planning?

Answer :

Function Point Analysis (FPA) is a software estimation technique used to measure the **functional size of a software system** based on the functionality provided to the user, rather than the amount of source code.

Purpose of FPA

1. Estimate Project Size

FPA helps determine the size of a software project by counting its functional components.

2. Estimate Development Effort

The calculated function points can be used to estimate the amount of effort required to develop the system.

3. Estimate Cost

Project managers can use the estimated effort to determine the approximate **development cost**.

4. Estimate Schedule

FPA helps estimate how much time may be required to complete the project.

5. Support Resource Planning

It helps determine the required number of developers, testers, and other resources.

6. Compare Different Projects

Function points provide a standardized measure that can be used to compare the size and productivity of different software projects.

2. List and explain the five basic components used in calculating Unadjusted Function Points (UFP).

Answer :

Unadjusted Function Points (UFP) are calculated by counting five basic types of user-recognizable functions in a software system. These are classified according to their **complexity** as low, average, or high.

1. External Inputs (EI)

External Inputs are processes through which **data enters the system from outside.**

Examples:

- User registration
- Adding a new book
- Entering customer details
- Placing an order

Purpose: To maintain or update internal data.

2. External Outputs (EO)

External Outputs are processes that **send processed information or results outside the system.**

Examples:

- Generating a sales report
- Displaying an invoice
- Printing a library report
- Sending an order confirmation

Purpose: To provide information or results to the user or another system.

3. External Inquiries (EQ)

External Inquiries involve **requesting information from the system without significantly updating or processing the stored data.**

Examples:

- Searching for a book
- Checking product availability
- Checking order status
- Viewing a customer's account

Purpose: To retrieve information from the system.

4. Internal Logical Files (ILF)

Internal Logical Files are **logical groups of data maintained within the application.**

Examples:

- Customer database
- Product database
- Employee records
- Order records
- Library book records

Purpose: To store and maintain data used by the application.

5. External Interface Files (EIF)

External Interface Files are **data files maintained by another application but accessed or referenced by the system.**

Examples:

- External payment gateway data
- External authentication database
- Government tax database
- External shipping service data

Purpose: To allow the application to use data maintained by an external system.

3. Differentiate between Internal Logical Files (ILF) and External Interface Files (EIF) with examples.

Answer :

Internal Logical File (ILF)	External Interface File (EIF)
Data is maintained by the application being measured.	Data is maintained by another application.
The application can create, update, and delete the data.	The application generally only reads or references the data.
It is internally maintained.	It is externally maintained.
Used to store data required by the application.	Used to access data maintained outside the application.
Example: Customer database maintained by an Online Shopping System.	Example: Payment information accessed from an external payment system.

4. Define Value Adjustment Factor (VAF). How is it used in calculating the final Function Points?

Answer :

The **Value Adjustment Factor (VAF)** is a factor used in **Function Point Analysis (FPA)** to adjust the **Unadjusted Function Points (UFP)** based on the overall technical and operational characteristics of the software system.

It considers **14 General System Characteristics (GSCs)** such as performance, security, data communications, reusability, and ease of installation.

Calculation of VAF

Each of the 14 characteristics is given a rating from **0 to 5**:

- **0** = No influence
- **1** = Incidental influence
- **2** = Moderate influence
- **3** = Average influence
- **4** = Significant influence
- **5** = Strong influence

The ratings are added to obtain the **Total Degree of Influence (TDI)**.

The VAF is calculated as:

$$\text{VAF} = 0.65 + (0.01 \times \text{TDI})$$

where:

TDI = Sum of the ratings of all 14 General System Characteristics

Since TDI ranges from 0 to 70:

$$0.65 \leq \text{VAF} \leq 1.35$$

Calculation of Final Function Points

Once UFP and VAF are known:

$$\text{FP} = \text{UFP} \times \text{VAF}$$

5. Why is it important to estimate software size before starting a project?

Answer :

1. **Accurate Effort Estimation**
Knowing the approximate size of the software helps estimate how much development effort will be required.
 2. **Cost Estimation**
Software size helps project managers calculate the expected **development and maintenance costs**.
 3. **Schedule Planning**
It helps estimate how much time will be needed to complete the project and set realistic deadlines.
 4. **Resource Allocation**
Managers can determine the required number of **developers, testers, tools, and infrastructure**.
 5. **Better Project Planning**
Size estimation provides a basis for creating a realistic project plan and assigning tasks.
 6. **Risk Management**
Large or complex projects can be identified early, allowing potential risks to be analyzed and managed.
 7. **Tracking Project Progress**
Estimated size can be compared with actual progress to determine whether the project is on schedule.
 8. **Improves Decision-Making**
Organizations can decide whether a project is **feasible, affordable, and achievable** before investing significant resources.
-

Practical No.14: Calculate cost of the project using COCOMO (Constructive Cost Model) / COCOMO II approach.

Conclusion : Successfully calculated cost of the project using COCOMO (Constructive Cost Model) / COCOMO II approach.

Practical related questions**1. What is the purpose of using the COCOMO model in software project estimation?**

Answer :

1. Estimate Development Effort

COCOMO estimates the amount of effort required to develop the software, usually measured in **person-months**.

2. Estimate Development Time

It helps determine the approximate time required to complete the project.

3. Estimate Staffing Requirements

The estimated effort and duration can be used to determine the approximate number of people needed for the project.

4. Estimate Project Cost

Once effort and staffing requirements are known, the organization can estimate the overall development cost.

5. Support Project Planning

COCOMO provides quantitative estimates that help managers prepare realistic schedules and allocate resources.

6. Compare Development Alternatives

Different project sizes or development approaches can be evaluated using estimated effort and time.

2. Differentiate between Basic COCOMO and COCOMO II.

Answer :

Basic COCOMO	COCOMO II
Developed for traditional software development environments.	Developed for modern software development environments.
Mainly uses KLOC (thousand lines of code) as the size measure.	Can use KSLOC, Function Points, Object Points , and other size measures depending on the model.
Uses a relatively simple estimation approach.	Uses more detailed estimation factors and scaling factors.
Has three development modes : Organic, Semi-Detached, and Embedded.	Uses different submodels such as Application Composition, Early Design, and Post-Architecture .
Less suitable for iterative, component-based, and modern development.	Better suited for iterative, incremental, component-based, and modern development .
Uses fixed project-mode coefficients.	Uses scale factors and effort multipliers to provide more flexible estimates.
Requires less project information.	Can incorporate more detailed project and development information.
Best suited for relatively straightforward estimation.	Better suited for complex and evolving software projects.

3. What are the key factors affecting cost estimation in COCOMO II?

Answer :

1. Software Size

- The estimated size of the software, usually measured in **KSLOC (thousand source lines of code)** or converted from Function Points/Object Points.
- Larger software generally requires more effort and cost.

2. Scale Factors

COCOMO II uses **five scale factors** that affect the overall effort:

- **PREC** – Precedentedness
- **FLEX** – Development flexibility
- **RESL** – Architecture/Risk resolution
- **TEAM** – Team cohesion
- **PMAT** – Process maturity

3. **Effort Multipliers**

COCOMO II uses **17 effort multipliers** in the Post-Architecture model. These represent factors such as:

- Required software reliability
- Product complexity
- Required reuse
- Platform difficulty
- Personnel capability
- Personnel experience
- Use of software tools
- Required development schedule

4. **Required Software Reliability**

Software requiring high reliability, such as safety-critical systems, generally requires more development effort.

5. **Product Complexity**

More complex software requires additional design, development, testing, and maintenance effort.

6. **Personnel Capability and Experience**

Highly skilled and experienced developers generally require less effort than less experienced developers for the same task.

7. **Development Environment and Tools**

Better development tools, automation, and modern environments can reduce development effort.

8. **Required Development Schedule**

A very tight schedule can increase effort and cost because additional resources or overtime may be required.

9. **Process Maturity**

A mature and well-defined development process generally improves productivity and reduces project risks.

4. **Define the term KLOC and explain its importance in cost estimation.**

Answer :

KLOC stands for **Kilo Lines of Code**. It represents the size of a software program in terms of **thousands of lines of source code**.

For example:

- **1 KLOC = 1,000 lines of code**
- **10 KLOC = 10,000 lines of code**
- **50 KLOC = 50,000 lines of code**

Importance of KLOC in Cost Estimation

1. **Measures Software Size**
KLOC provides a numerical estimate of the size of a software project.
2. **Estimates Development Effort**
Models such as **COCOMO** use KLOC to estimate the effort required, usually in person-months.
3. **Estimates Development Time**
Once the effort is estimated, the development duration can also be calculated.
4. **Helps Estimate Cost**
The estimated effort and staffing requirements can be used to calculate the approximate project cost.
5. **Helps Resource Planning**
KLOC-based estimates help determine the number of developers and other resources required.
6. **Supports Project Comparison**
KLOC can be used to compare the size and productivity of similar software projects.

5. Explain the significance of the Effort Multipliers in COCOMO II.

Answer :

1. **Improve Estimation Accuracy**
They adjust the effort estimate according to actual project conditions.
 2. **Consider Product Characteristics**
Factors such as **required reliability and software complexity** can increase or decrease the required effort.
 3. **Consider Personnel Capability**
The skill, experience, and capability of developers affect productivity and therefore effort.
 4. **Consider Development Environment**
Tools, platform constraints, and development facilities can influence the amount of effort required.
 5. **Consider Project Requirements**
Factors such as required development schedule and level of software reuse can affect effort.
 6. **Support More Realistic Planning**
By considering project-specific conditions, managers can make better decisions about **cost, schedule, and staffing**.
-

Practical No.15: Create software project scheduling charts using CPM (Critical Path Method) / PERT (Project Evaluation and Review Technique).

Conclusion : Successfully created software project scheduling charts using CPM (Critical Path Method) / PERT (Project Evaluation and Review Technique).

Practical related questions

1. What is the difference between CPM and PERT techniques?

Answer :

CPM	PERT
CPM stands for Critical Path Method.	PERT stands for Program Evaluation and Review Technique.
Uses deterministic activity times .	Uses probabilistic activity times .
Assumes activity duration is known or can be estimated with reasonable certainty.	Uses three time estimates : optimistic, most likely, and pessimistic.
Mainly focuses on time and cost optimization .	Mainly focuses on time uncertainty and risk .
Suitable for projects where activity durations are relatively predictable.	Suitable for projects where activity durations are uncertain.
Usually uses a single time estimate for each activity.	Uses three time estimates for each activity.
Useful for construction, manufacturing, and repetitive projects .	Useful for research, development, and new software projects .

2. How is the expected time (TE) calculated in PERT?

Answer :

In **PERT (Program Evaluation and Review Technique)**, the **Expected Time (TE)** of an activity is calculated using three time estimates:

- **Optimistic Time (O):** Minimum time required if everything goes well.
- **Most Likely Time (M):** Normal expected time under usual conditions.
- **Pessimistic Time (P):** Maximum time required if significant problems occur.

Formula

$$T_E = \frac{O + 4M + P}{6}$$

6

The **most likely time is given four times the weight** because it is considered the most realistic estimate.

Example

Suppose a software development activity has:

- **O = 2 days**
- **M = 5 days**
- **P = 8 days**

Then:

$$T_E = \frac{2 + 4(5) + 8}{6}$$

$$T_E = \frac{30}{6}$$

$$T_E = 5 \text{ days}$$

3. What is a critical path and why is it important in project scheduling?

Answer :

The **Critical Path** is the **longest sequence of dependent activities** in a project network that determines the **minimum time required to complete the entire project**.

Activities on the critical path have **zero total float (slack)**. Therefore, if any critical activity is delayed, the **entire project completion date is delayed**, unless corrective action is taken.

Importance of Critical Path

1. **Determines Project Duration**

It identifies the minimum time required to complete the project.

2. **Identifies Critical Activities**

It shows which activities cannot be delayed without affecting the project completion date.

3. **Helps in Resource Allocation**

Managers can prioritize resources for critical activities.

4. **Improves Schedule Management**

It helps managers focus on activities that have the greatest effect on the project schedule.

5. **Helps Identify Delays Early**

Monitoring critical activities allows potential schedule delays to be detected quickly.

6. **Supports Decision-Making**

Managers can decide where to add resources or take corrective actions to reduce delays.

4. How do you determine slack time for an activity in a network diagram?

Answer :

Slack time (or float) is the amount of time an activity can be delayed **without delaying the overall project completion time**.

Formula

Slack time can be calculated as:

$$\text{Slack} = \text{LS} - \text{ES}$$

or

$$\text{Slack} = \text{LF} - \text{EF}$$

Where:

- **ES (Earliest Start):** Earliest time the activity can start.
- **EF (Earliest Finish):** Earliest time the activity can finish.
- **LS (Latest Start):** Latest time the activity can start without delaying the project.
- **LF (Latest Finish):** Latest time the activity can finish without delaying the project.

Steps to Determine Slack

1. Calculate the **Earliest Start (ES)** and **Earliest Finish (EF)** using a **forward pass**.
 2. Calculate the **Latest Start (LS)** and **Latest Finish (LF)** using a **backward pass**.
 3. Apply the formula : **Slack = LS - ES**
-

Practical No.16: Track progress of the project using Timeline Charts/ Gantt charts.

Conclusion : Successfully tracked progress of the project using Timeline Charts/ Gantt charts.

Practical related questions**1. What is a Gantt chart and how is it useful in project management?**

Answer :

A **Gantt Chart** is a **visual project management tool** that represents project activities or tasks against a **timeline**. Each task is shown as a horizontal bar, where the position and length of the bar indicate the **start date, duration, and end date** of the task.

Example

Task	Duration	Timeline
Requirement Analysis	3 days	
System Design	4 days	
Coding	6 days	
Testing	3 days	

A typical Gantt chart can also show **dependencies, milestones, progress, and assigned resources**.

Uses of Gantt Charts in Project Management

- 1. Project Scheduling**
Helps plan when each task should start and finish.
- 2. Tracks Project Progress**
Managers can compare actual progress with the planned schedule.
- 3. Shows Task Dependencies**
Makes it easier to understand which tasks must be completed before others can begin.
- 4. Helps Manage Deadlines**
Important deadlines and milestones can be clearly displayed.
- 5. Resource Planning**
Helps managers determine when developers, testers, and other resources are needed.
- 6. Identifies Delays**
Delayed tasks can be easily identified and corrective action can be taken.
- 7. Improves Communication**
Provides a simple visual representation that can be shared with team members and stakeholders.

2. How do you identify task dependencies in a Gantt chart?

Answer :

In a Gantt chart, dependencies are usually identified using **connecting lines or arrows** between task bars.

Steps to Identify Task Dependencies

1. List All Project Tasks

Identify all activities required to complete the project.

2. Determine the Sequence

Decide which tasks need to be completed before other tasks can begin.

3. Identify the Relationship Between Tasks

Common dependency types include:

- **Finish-to-Start (FS):** Task B starts after Task A finishes.
- **Start-to-Start (SS):** Task B starts after Task A starts.
- **Finish-to-Finish (FF):** Task B finishes after Task A finishes.
- **Start-to-Finish (SF):** Task B finishes after Task A starts.

4. Connect the Tasks

Draw arrows or dependency lines between the related task bars in the Gantt chart.

3. Which software tools can be used to create a Gantt chart?

Answer :

1. Microsoft Project
2. Microsoft Excel
3. Google Sheets
4. Smartsheet
5. Primavera P6
6. Trello
7. Asana
8. ClickUp

4. How does a timeline chart differ from a Gantt chart?

Answer :

Timeline Chart	Gantt Chart
Shows events or milestones in chronological order .	Shows tasks and their durations across a timeline.
Mainly focuses on when events occur .	Focuses on when tasks start, finish, and how long they take .
Usually simpler and more visual.	More detailed and structured for project scheduling.
Does not necessarily show task dependencies.	Can show task dependencies using arrows or connecting lines.
Useful for presenting major events, milestones, and deadlines .	Useful for planning, scheduling, and monitoring project activities .
Usually does not show resource allocation or task progress in detail.	Can show progress, resources, milestones, and dependencies .
Example: Project milestones from January to June.	Example: Requirements → Design → Coding → Testing with their durations.

Practical No.17: Prepare SQA plan that facilitates various attributes of quality of process.

Conclusion : Successfully prepared SQA plan that facilitates various attributes of quality of process.

Practical related questions

1. List and describe the quality control activities in the SQA plan of a Library Management System?

Answer :

1. Requirement Review

- Review the SRS to ensure that requirements are **correct, complete, consistent, and testable**.
- Example: Verify that requirements for book issue, return, reservation, and fine calculation are clearly defined.

2. Design Review

- Examine the system design to identify design errors before coding begins.
- Check database design, user interfaces, system architecture, and module relationships.

3. Code Review

- Developers or reviewers inspect source code to identify **coding errors, security issues, and violations of coding standards**.
- Example: Check the code responsible for calculating overdue fines.

4. Unit Testing

- Individual modules are tested separately to verify that each module works correctly.
- Example: Test the book-search module independently.

5. Integration Testing

- Tests whether different modules work correctly when combined.
- Example: Verify that the book-issue module correctly updates the book availability database.

6. System Testing

- Tests the complete Library Management System against the specified requirements.
- Functions such as login, search, issue, return, reservation, and fine calculation are tested together.

7. Regression Testing

- Re-tests existing functionality after changes or bug fixes to ensure that previously working features have not been affected.

8. Performance Testing

- Checks the system's response time, speed, and behavior under different workloads.
- Example: Verify that book searches remain responsive when many users access the system simultaneously.

9. Security Testing

- Ensures that unauthorized users cannot access or modify sensitive information.
- Example: Only authorized librarians should be able to add or remove book records.

10. Defect Tracking and Management

- Identified defects are recorded, assigned to developers, fixed, and retested.
- Each defect can be tracked from **identification to closure**.

11. User Acceptance Testing (UAT)

- Actual users, such as librarians or administrators, verify that the system meets their practical requirements.

12. Documentation Review

- Checks user manuals, technical documents, test reports, and other documentation for correctness and completeness.

2. What are the key differences between preventive and corrective quality assurance activities? Give examples from your SQA plan.

Answer :

Preventive QA Activities	Corrective QA Activities
Focus on preventing defects .	Focus on finding and fixing defects .
Usually performed before or during development .	Usually performed after a defect is detected .
Reduce the probability of errors.	Reduce the impact of existing errors.
Focus on improving the process and development practices .	Focus on correcting the software/product .
Examples: requirement reviews, design reviews, coding standards, developer training.	Examples: defect fixing, debugging, retesting, regression testing.

Examples from a Library Management System SQA Plan

Preventive Activities

1. Requirement Review

- Review the SRS before development to identify incomplete or ambiguous requirements.
- Example: Clearly define how overdue fines should be calculated.

2. Design Review

- Review the database and system design before coding.
- Example: Ensure that relationships between **Books, Members, and Issue Records** are correctly designed.

3. Coding Standards

- Establish coding rules to reduce programming errors.
- Example: Use consistent naming conventions and proper error handling.

4. Developer Training

- Train developers on security, testing, and project standards to reduce the occurrence of defects.

Corrective Activities

1. Defect Tracking

- Record defects discovered during testing and assign them to developers for correction.
- Example: Record an error where the system calculates an incorrect fine.

2. Debugging and Code Fixing

- Analyze the cause of the defect and modify the code to correct it.

3. Retesting

- Test the corrected functionality to confirm that the defect has been fixed.
- Example: Recalculate the fine after fixing the fine-calculation module.

4. Regression Testing

- Test related functions after a fix to ensure that the correction has not introduced new defects.

3. List any three quality attributes that should be considered while preparing an SQA plan.

Answer :

While preparing an **SQA (Software Quality Assurance) plan**, several quality attributes should be considered to ensure that the software meets user and business requirements.

1. Reliability

Reliability refers to the ability of the software to perform its required functions **correctly and consistently** for a specified period.

Example:

A Library Management System should correctly issue and return books without losing or corrupting transaction records.

2. Performance

Performance refers to how efficiently the software responds to user requests and handles workload.

Example:

The system should display search results quickly, even when many books and users are stored in the database.

3. Security

Security refers to protecting the software and its data from **unauthorized access, modification, or misuse**.

Example:

Only authorized librarians should be allowed to add, update, or delete book records.

4. What is the purpose of an SQA plan in software development?

Answer :

1. **Ensure Software Quality**

It ensures that the software meets specified **functional and non-functional requirements**.

2. **Define Quality Standards**

It specifies the coding standards, development processes, testing standards, and other quality criteria to be followed.

3. **Prevent and Detect Defects**

It defines preventive activities such as reviews and corrective activities such as testing and debugging.

4. **Define Testing Activities**

It specifies what types of testing will be performed, such as:

- Unit testing
- Integration testing
- System testing
- Regression testing
- Acceptance testing

5. **Assign Responsibilities**

It identifies who is responsible for activities such as testing, code reviews, defect tracking, and quality monitoring.

6. **Monitor Project Quality**

It establishes methods for measuring and monitoring software quality throughout development.

7. **Manage Defects**

It defines how defects will be **reported, tracked, corrected, verified, and closed**.

8. **Ensure Process Compliance**

It helps ensure that the development team follows the defined processes, standards, and organizational guidelines.

9. **Reduce Project Risks**

Early quality checks and reviews help reduce the risk of major defects, delays, and failures.

Practical No.18: Prepare SQA plan that facilitates various attributes of quality of product.

Conclusion : Successfully prepared SQA plan that facilitates various attributes of quality of product.

Practical related questions

1. What are the key quality attributes considered in your SQA plan and why?

Answer :

Quality Attribute	Why It Is Important	Example in Library Management System
Reliability	Ensures the system performs correctly and consistently without failures.	Book issue and return transactions should be recorded correctly.
Performance	Ensures the system responds quickly and handles the expected workload.	Book searches should return results quickly even with a large database.
Security	Protects user and library data from unauthorized access.	Only authorized librarians can modify book records.
Usability	Makes the system easy for librarians and members to learn and use.	Users should easily find books and check their availability.
Maintainability	Makes it easier to modify, update, and fix the system.	New features such as online reservations can be added without affecting existing functions.
Availability	Ensures the system is accessible when users need it.	Librarians should be able to access the system during library operating hours.
Scalability	Ensures the system can handle growth in users, books, and transactions.	The system should continue working efficiently as the library database grows.

2. How does your SQA plan ensure the reliability of the software?

Answer :

An **SQA (Software Quality Assurance) plan** ensures software reliability by defining systematic activities to **prevent defects, detect failures, and verify that the software performs consistently** under expected conditions.

For a **Library Management System**, reliability can be ensured through the following activities:

1. Requirement Reviews

The SQA team reviews requirements to ensure they are **correct, complete, consistent, and testable**.

Example: Clearly defining rules for book issue, return, reservation, and fine calculation.

2. Design Reviews

The system architecture, database design, and module interactions are reviewed before implementation to identify potential problems early.

3. Code Reviews

Source code is reviewed to identify programming errors, incorrect logic, and poor coding practices before the software is tested.

4. Thorough Testing

Different levels of testing are performed:

- **Unit Testing** – Tests individual modules.
- **Integration Testing** – Tests interactions between modules.
- **System Testing** – Tests the complete system.
- **Regression Testing** – Ensures that changes do not break existing functionality.

5. Defect Tracking and Correction

All identified defects are recorded, prioritized, fixed, and verified through **retesting**.

6. Performance Testing

The system is tested under different workloads to ensure it remains stable and responsive when many users access it simultaneously.

7. Data Backup and Recovery Testing

Regular backups and recovery procedures are tested to prevent permanent loss of important library data.

8. Security Testing

Authentication and authorization mechanisms are tested to prevent unauthorized users from modifying or accessing sensitive information.

9. User Acceptance Testing

Librarians and other intended users verify that the system works correctly in real-world scenarios.

10. Continuous Monitoring

After deployment, system performance, errors, and failures are monitored so that problems can be identified and corrected quickly.

3. Explain the role of testing in achieving software quality.

Answer :

1. Detects Defects

- Testing identifies errors and defects before the software is delivered to users.
- Example: Detecting an incorrect fine calculation in a Library Management System.

2. Validates Requirements

- Testing verifies whether the software performs according to the requirements defined in the **SRS**.
- Example: Checking whether users can successfully issue and return books.

3. Improves Reliability

- Repeated testing under different conditions ensures that the system behaves consistently and reduces unexpected failures.

4. Ensures Functional Correctness

- Testing confirms that individual features produce the expected results.
- Example: Searching for a book should display the correct book details and availability.

5. Checks Performance

- Performance testing determines whether the system responds within acceptable time limits and can handle the expected number of users.

6. Improves Security

- Security testing identifies vulnerabilities and ensures that unauthorized users cannot access restricted information.

7. Supports Regression Testing

- After modifications or bug fixes, regression testing ensures that existing functionality continues to work correctly.

8. Improves User Satisfaction

- A properly tested system is less likely to contain errors, providing users with a more reliable and positive experience.

9. Reduces Maintenance Cost

- Finding and fixing defects during development is generally less expensive than fixing them after deployment.

10. Supports Software Quality Assurance

- Testing provides measurable evidence that the software meets defined quality standards and requirements.

4. Why is documentation important in SQA planning?

Answer :

1. **Defines Quality Standards**
 - Documentation specifies the quality standards, procedures, and guidelines that the development team must follow.
 2. **Provides Clear Requirements**
 - Documents such as the **SRS** clearly describe what the software should do.
 - This helps testers create accurate test cases.
 3. **Ensures Consistency**
 - Documented processes ensure that team members follow the same procedures for development, testing, and defect management.
 4. **Supports Testing**
 - Test plans, test cases, and test reports provide a structured way to verify software functionality and quality.
 5. **Helps Track Defects**
 - Defect reports record the problem, its severity, steps to reproduce it, and its resolution.
 6. **Provides Traceability**
 - Documentation allows requirements to be traced to design, implementation, and test cases.
 - This helps ensure that **every requirement has been tested**.
 7. **Improves Communication**
 - Documents provide a common reference for developers, testers, project managers, and stakeholders.
 8. **Supports Maintenance**
 - Future developers can understand the system, its requirements, architecture, and previous defects more easily.
 9. **Provides Evidence of Quality**
 - Test results, review records, and quality reports provide evidence that the required quality activities were performed.
 10. **Supports Audits and Compliance**
 - Proper documentation helps organizations demonstrate that defined processes and quality standards were followed.
-